Computer and Assembly Language

CSE 4095

Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

Taken from Dr. Yifeng Zhu's Lecture Notes

Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 1

Embedded Systems

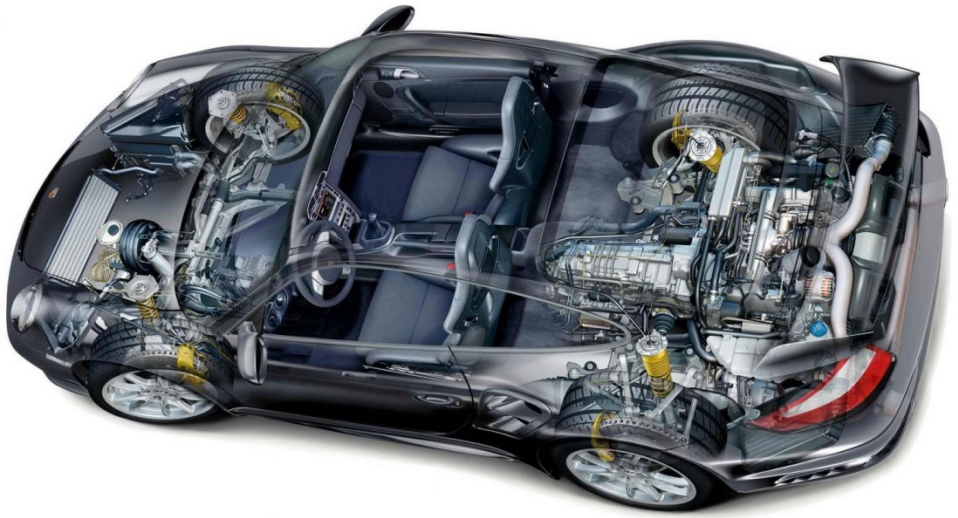












Why Learn Assembly Languages



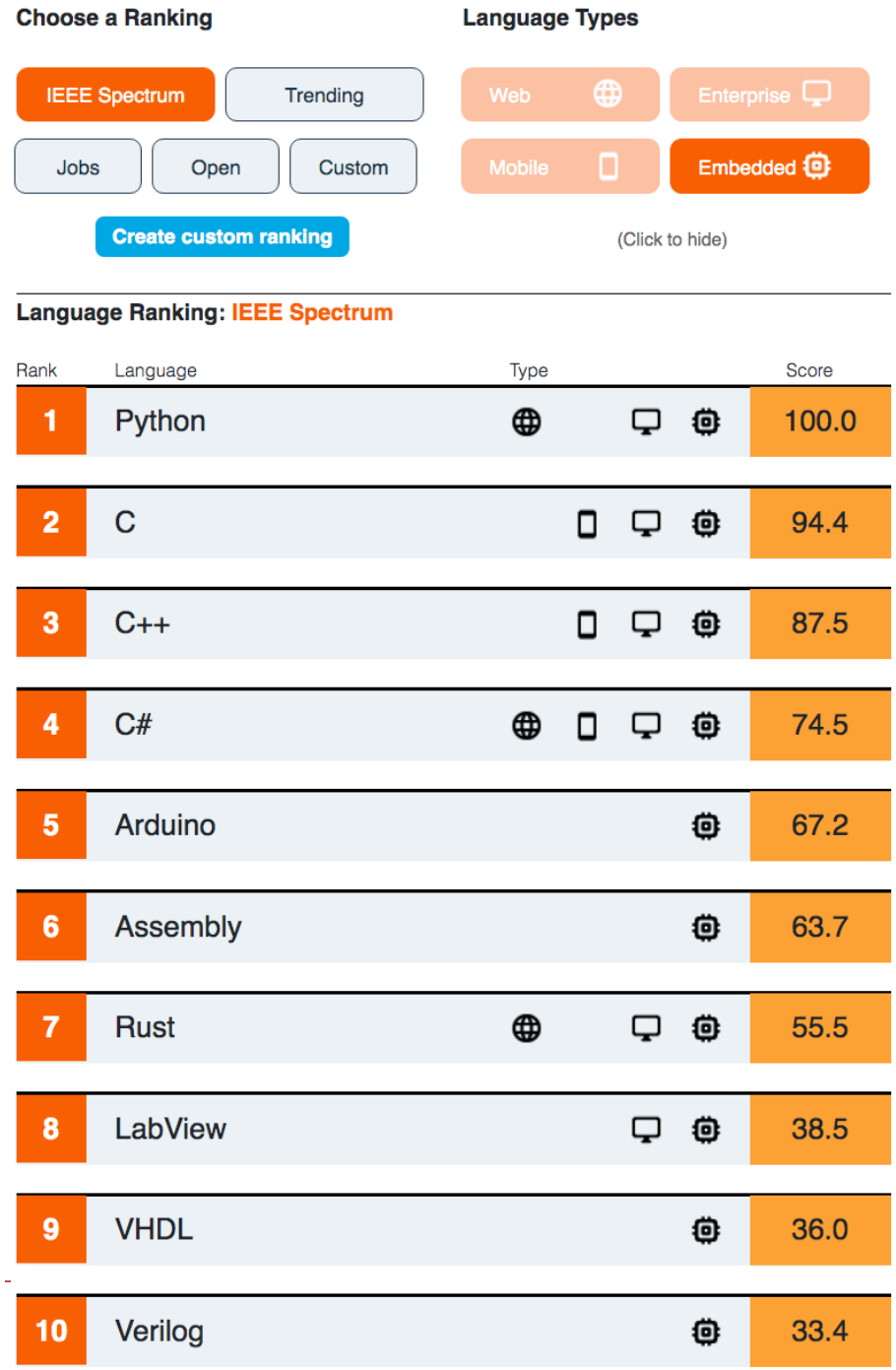
- Gain insights about what is under the hood of a processor
- Assembly should be used for performance critical sessions
- Assembly must be used for processor-dependent instructions that are not supported by compilers
- Understanding assembly helps us write better HLLs

http://www.andysinger.com/

IEEE 2019 Survey: Top Programming Languages for Embedded Systems

Overall Ranking

https://spectrum.ieee.org/static/interactive-the-top-programming-languages-2019



Choose a Ranking

IEEE Spectrum | Trending | Jobs | Open | Custom

Create custom ranking

Language Types

Web | Enterprise | Mobile | Embedded

(Click to hide)

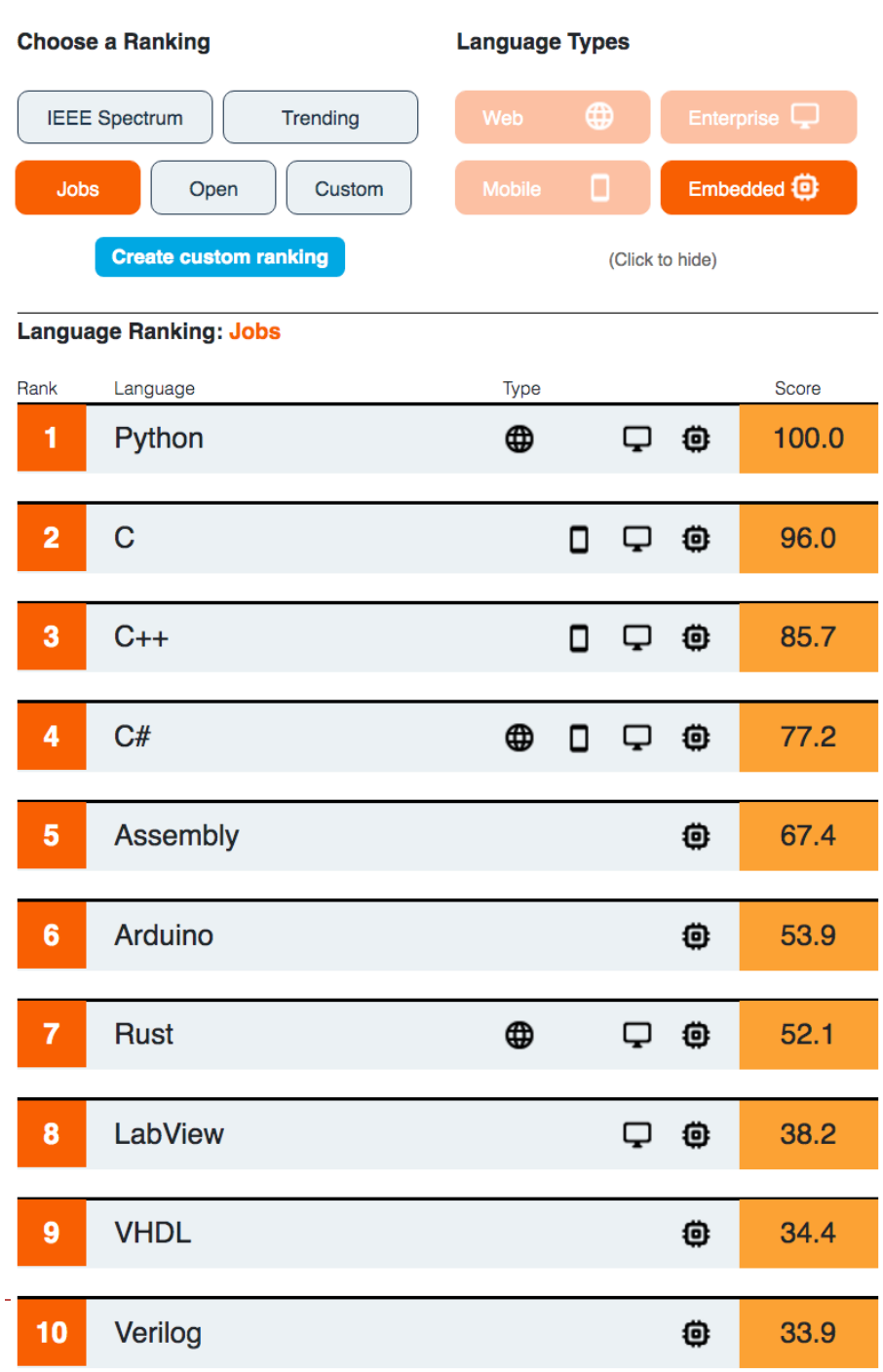
Language Ranking: IEEE Spectrum

Rank	Language	Type	Score
1	Python	web, desktop, embedded	100.0
2	C	mobile, desktop, embedded	94.4
3	C++	mobile, desktop, embedded	87.5
4	C#	web, mobile, desktop, embedded	74.5
5	Arduino	embedded	67.2
6	Assembly	embedded	63.7
7	Rust	web, desktop, embedded	55.5
8	LabView	desktop, embedded	38.5
9	VHDL	embedded	36.0
10	Verilog	embedded	33.4

IEEE 2019 Survey: Top Programming Languages for Embedded Systems

Job postings on the IEEE Job Site

https://spectrum.ieee.org/static/interactive-the-top-programming-languages-2019



Choose a Ranking

IEEE Spectrum | Trending

Jobs | Open | Custom

Create custom ranking

Language Types

Web | Enterprise

Mobile | Embedded

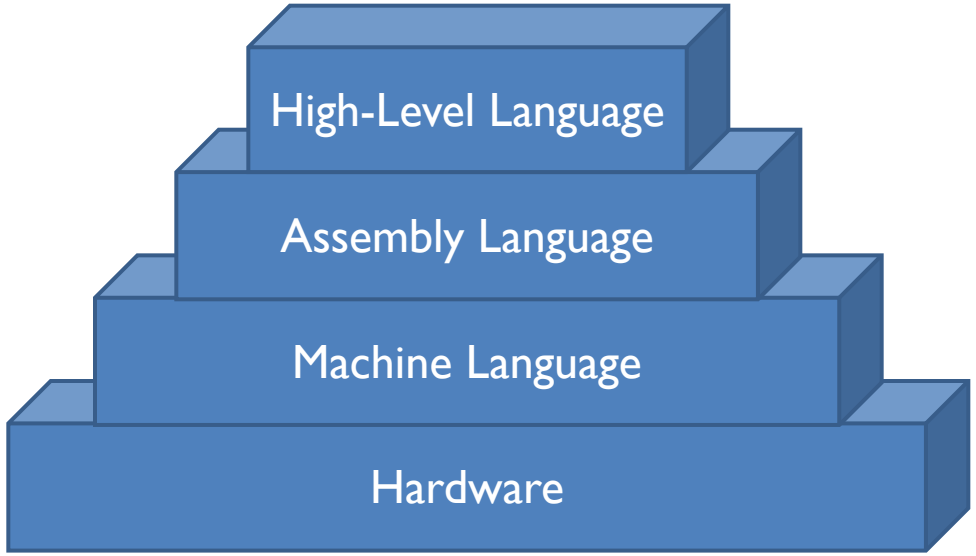
(Click to hide)

Language Ranking: Jobs

Rank	Language	Type	Score
1	Python	Web, Enterprise, Embedded	100.0
2	C	Mobile, Enterprise, Embedded	96.0
3	C++	Mobile, Enterprise, Embedded	85.7
4	C#	Web, Mobile, Enterprise, Embedded	77.2
5	Assembly	Embedded	67.4
6	Arduino	Embedded	53.9
7	Rust	Web, Enterprise, Embedded	52.1
8	LabView	Enterprise, Embedded	38.2
9	VHDL	Embedded	34.4
10	Verilog	Embedded	33.9

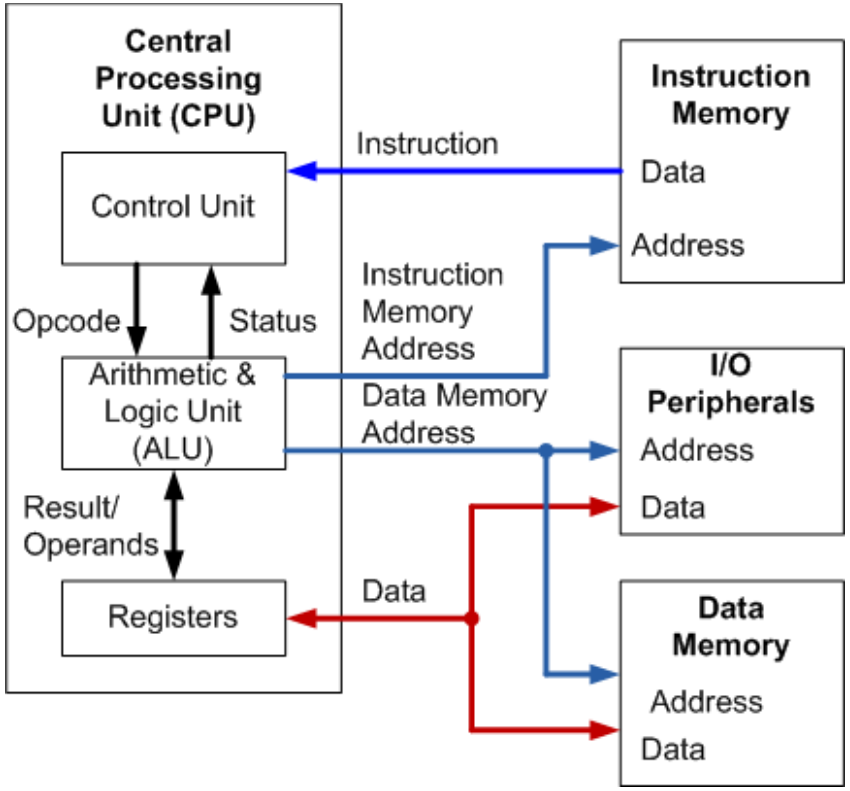
Assembly: Not just another language

- Assembly vs high level languages (HLLs).
  - Most embedded systems are programmed in HLLs
  - Assembly disadvantages
    - difficult to develop, read, and maintain
    - bug prone
    - not portable
- However, assembly isn't "just another language".
  - Interface between hardware and software
  - Implements high-level languages



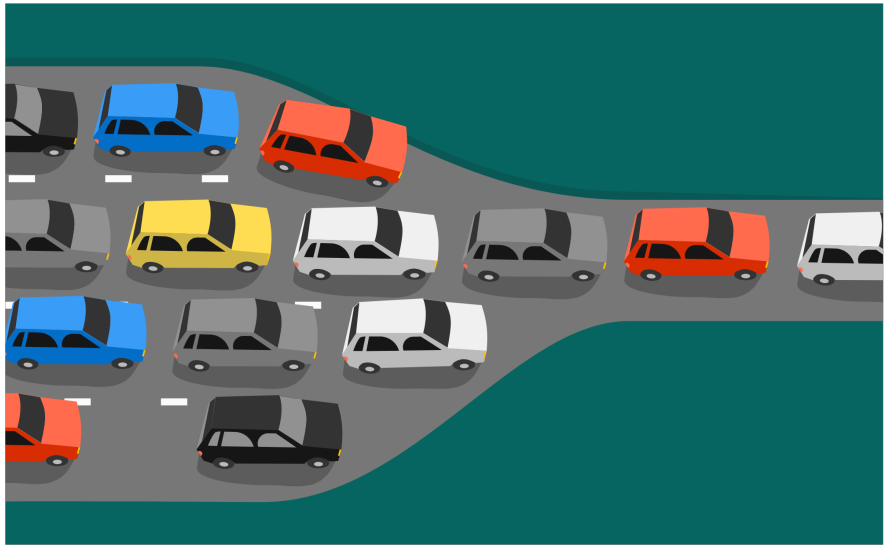
Assembly: Learn how processors work

- Learn about the inner workings of a processor
  - Data representation
  - Registers
  - Computer arithmetic
  - Memory addressing
  - Instruction set
  - I/O



Assembly: Faster and smaller

- Assembly program runs faster than HLLs.
  - Performance critical codes must be written in assembly.
  - Use the profiling tools to find the performance bottleneck and rewrite that code section in assembly
  - Latency-sensitive applications, such as aircraft controller
  - Some C compilers do not use some special Thumb instructions, such as ROR (Rotate Right) and RRX (Rotate Right Extended).



brilliant.org

- Cost-sensitive applications
  - Assembly consumes little memory
  - Embedded devices, where the size of code is limited, wash machine controller, automobile controllers

Assembly: The only choice sometimes

- Hardware/processor specific code,
  - Special instructions not supported by a compiler
    - **CPSID I** *; Disable IRQ by setting PRIMASK*
    - **CPSIE I** *; Enable IRQ by clearing PRIMASK*
    - **MSR/MSR** *; Read/write to special registers*
    - **WFI** *; Enter low-power & wait for interrupt*

```
// Enable Interrupts
__attribute__( ( always_inline ) )
static inline void __enable_irq(void)
{
    __asm("cpsie i");
}

// Disable Interrupts
__attribute__( ( always_inline ) )
static inline void __disable_irq(void)
{
    __asm("cpsid i");
}
```

cmsis_armcc.h

Assembly: Help you write better HLLs

```
uint32_t x = 1;
int32_t  y = -1;

if (x > y)
   printf("Of course.")
else
   printf("Something is wrong!");
```

Output:

Something is wrong!

```
r0 = 0x00000001   ; x = 1
r1 = 0xFFFFFFFF   ; y = -1
```

```
CMP r0, r1
BLS else   ; Branch on Unsigned Lower than or Same
```

```
compiling main.c...
linking...
Program Size: Code=640 RO-data=424 RW-data=8 ZI-data=5472
FromELF: creating hex file...
".\Objects\project.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:01
```

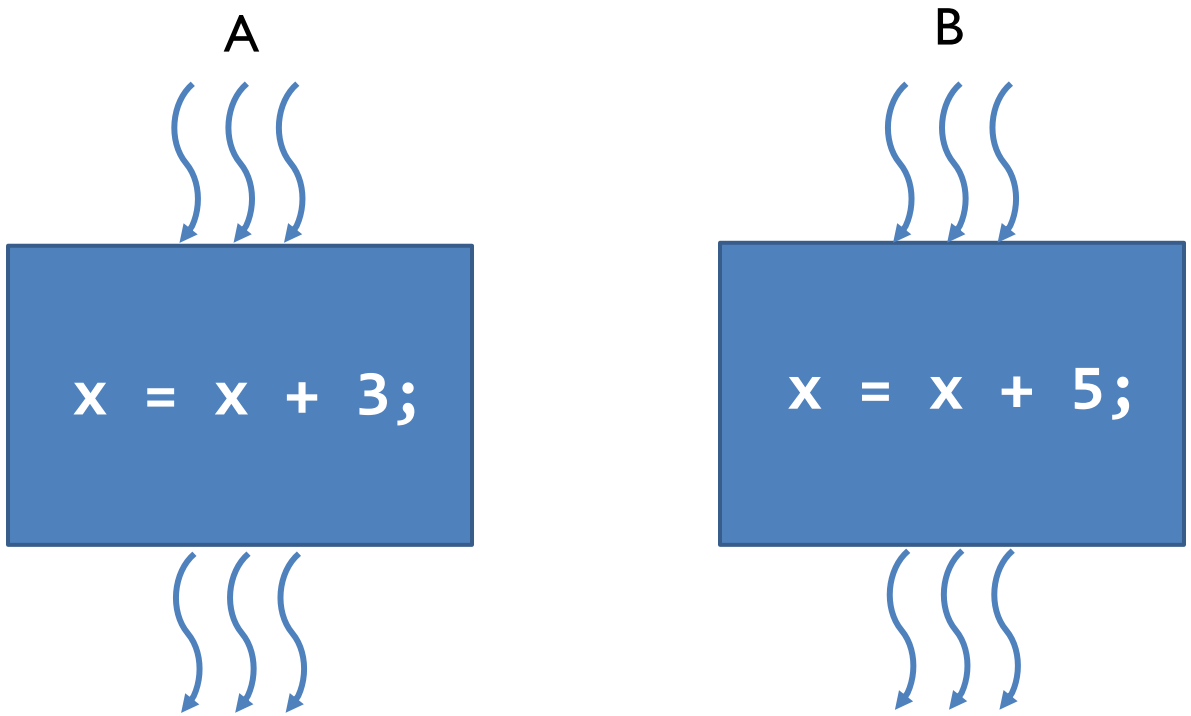
Assembly: Help you write better HLLs

```
uint32_t x = 1;
int32_t  y = -1;

if ((int32_t) x > y)
   printf("Of course.")
else
   printf("Something is wrong!");
```

Assembly: Help you write better HLLs

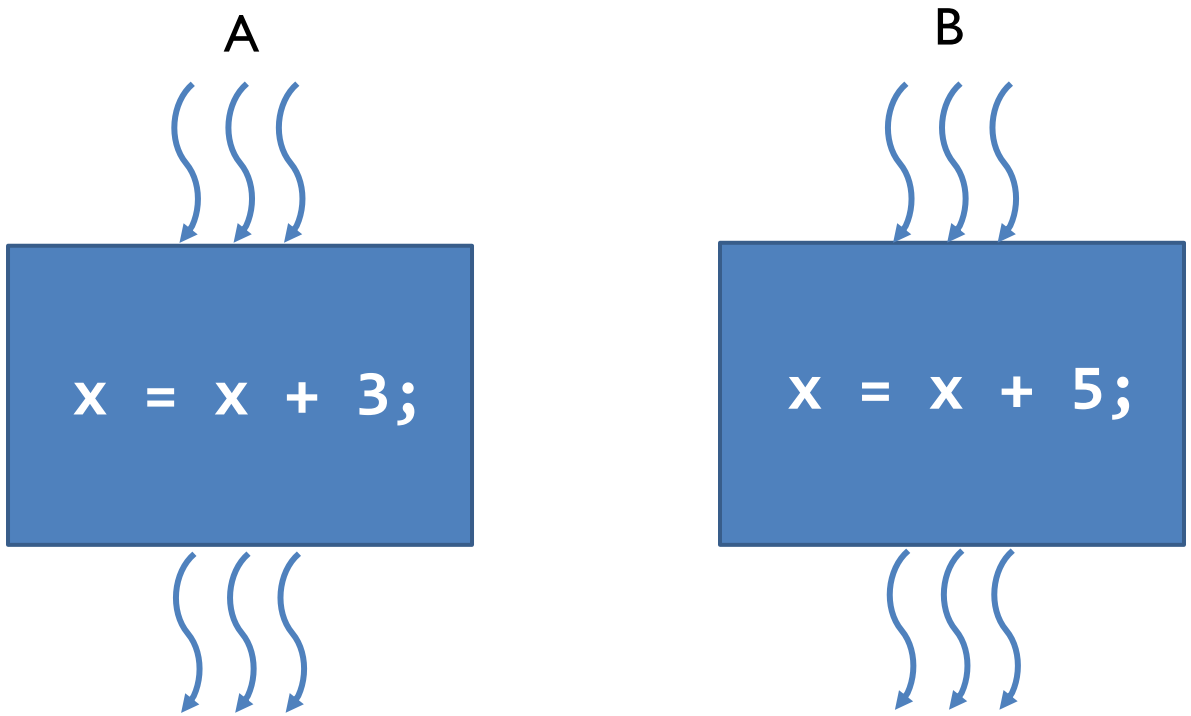
```
x = 1;
```



What is the final value of x?

Assembly: Help you write better HLLs

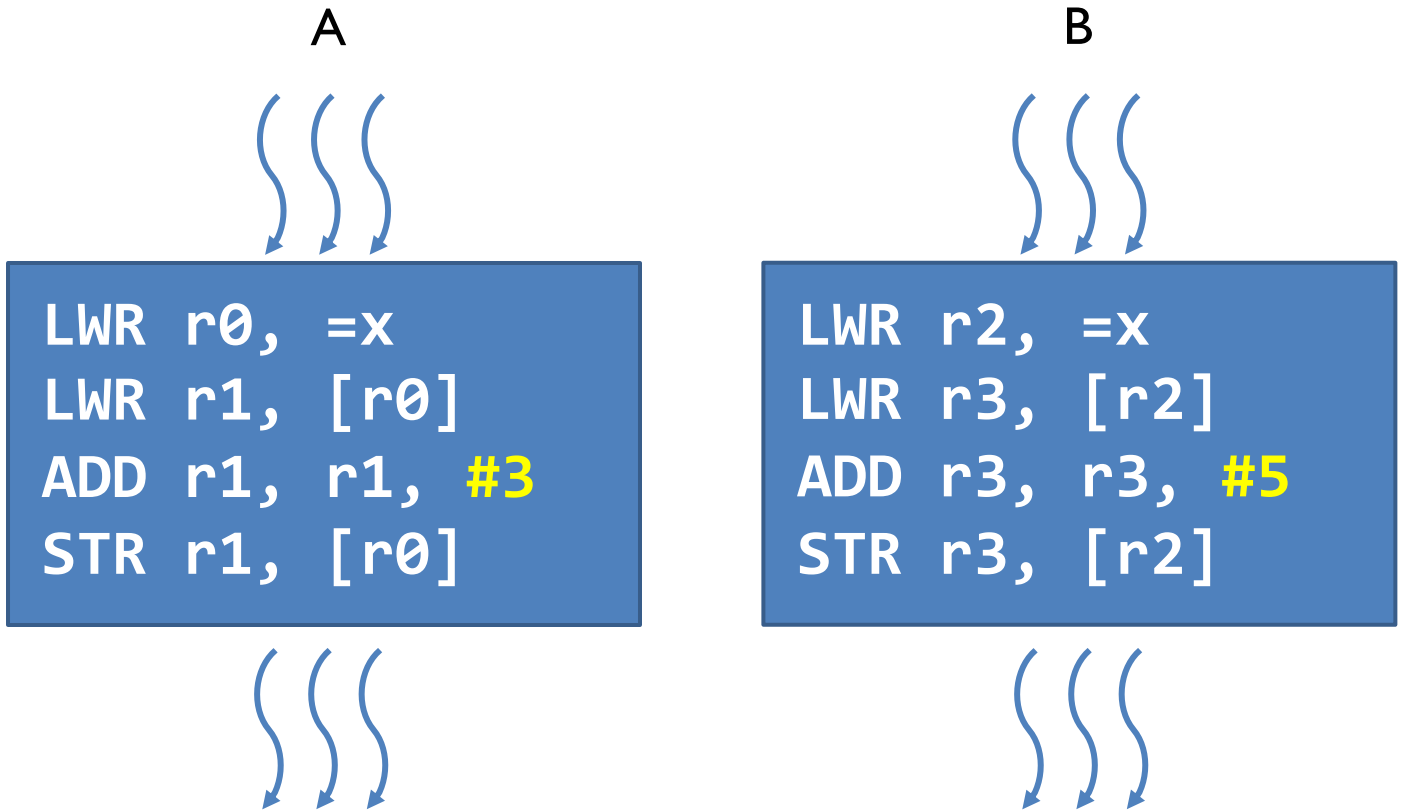
```
x = 1;
```




```
x = 4, 6, 9;
```

Assembly: Help you write better HLLs

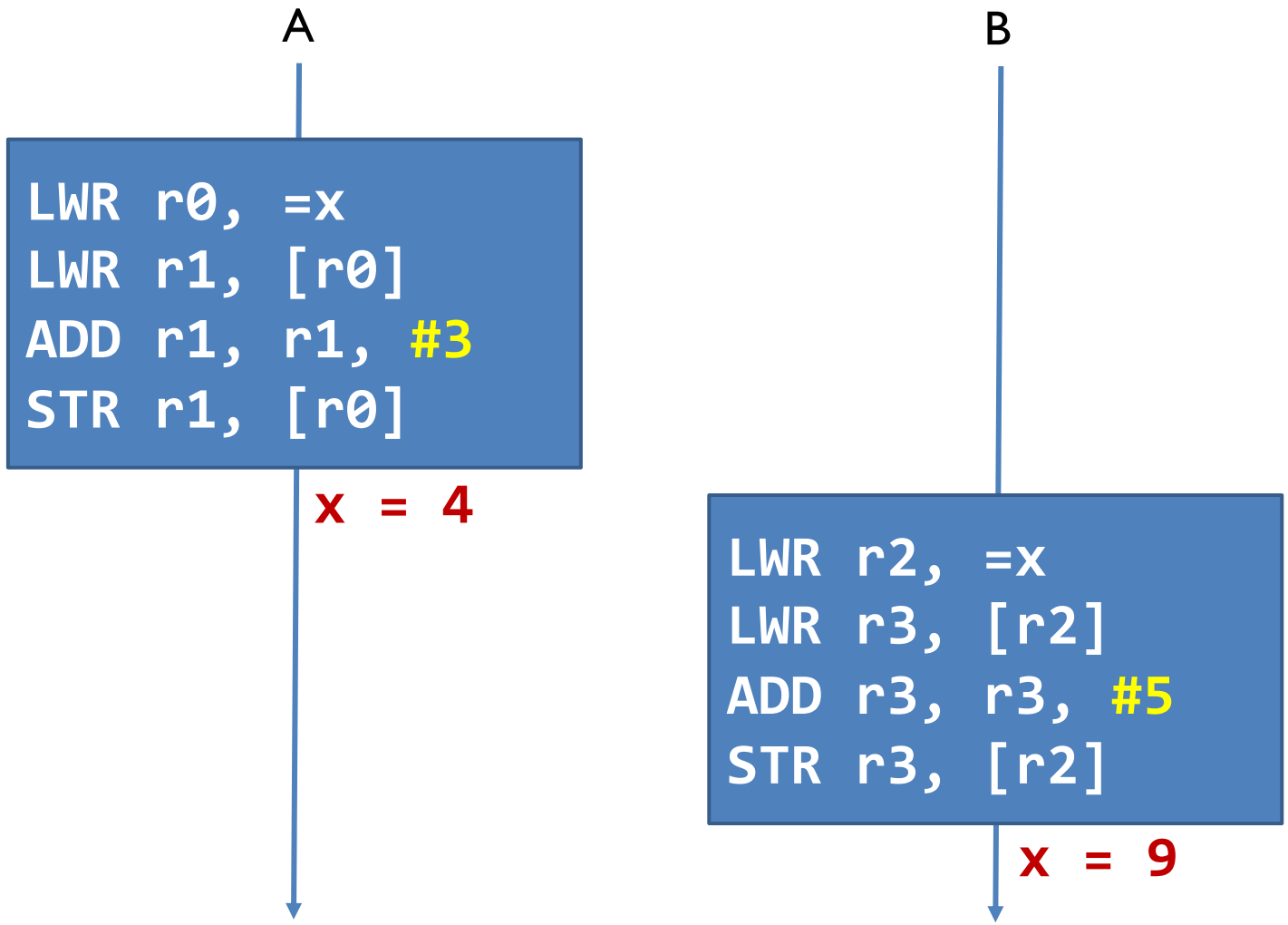
```
x = 1;
```




```
x = 4, 6, 9;
```

Assembly: Help you write better HLLs

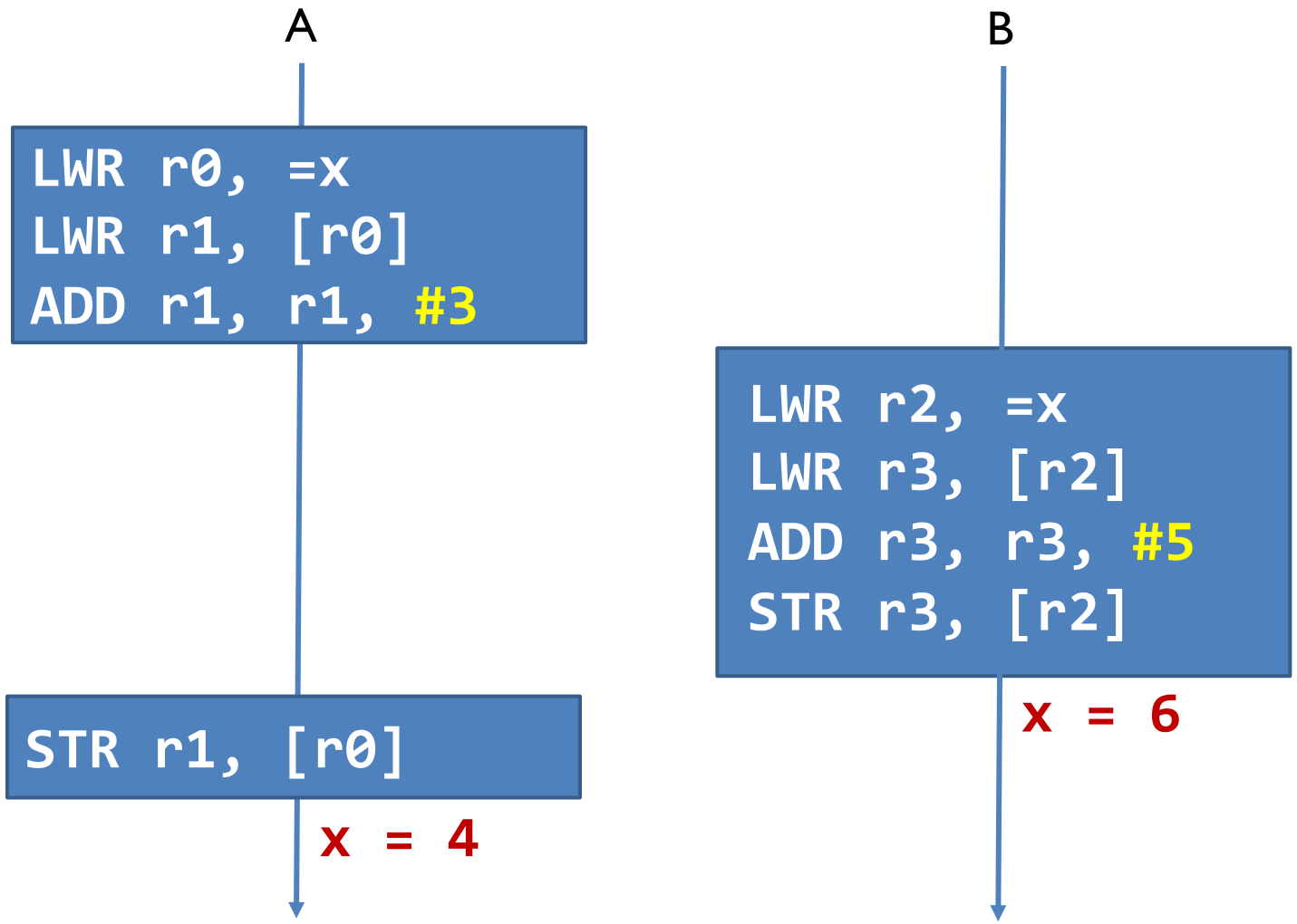
x = 1;



x = 9

Assembly: Help you write better HLLs

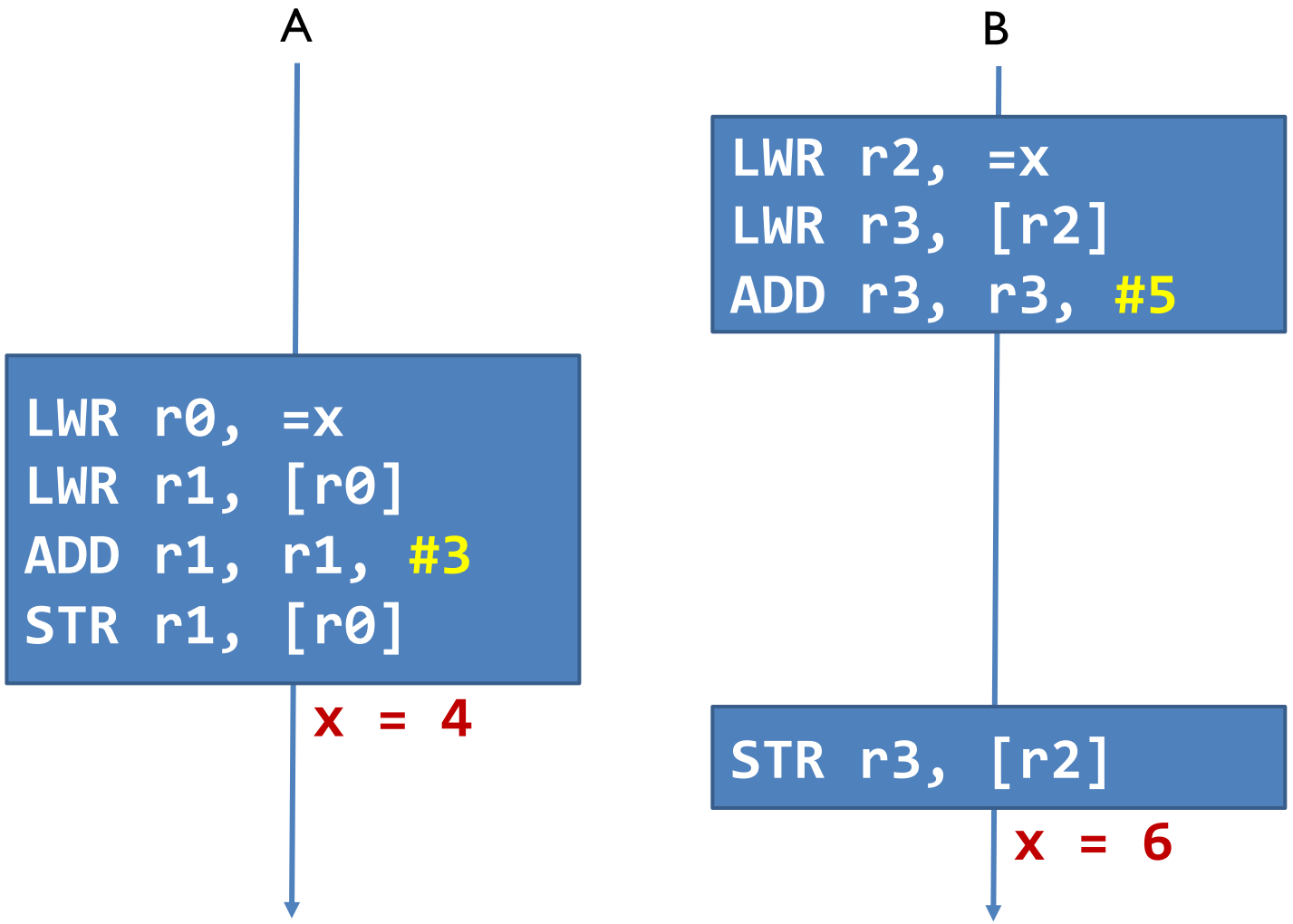
x = 1;



x = 4

Assembly: Help you write better HLLs

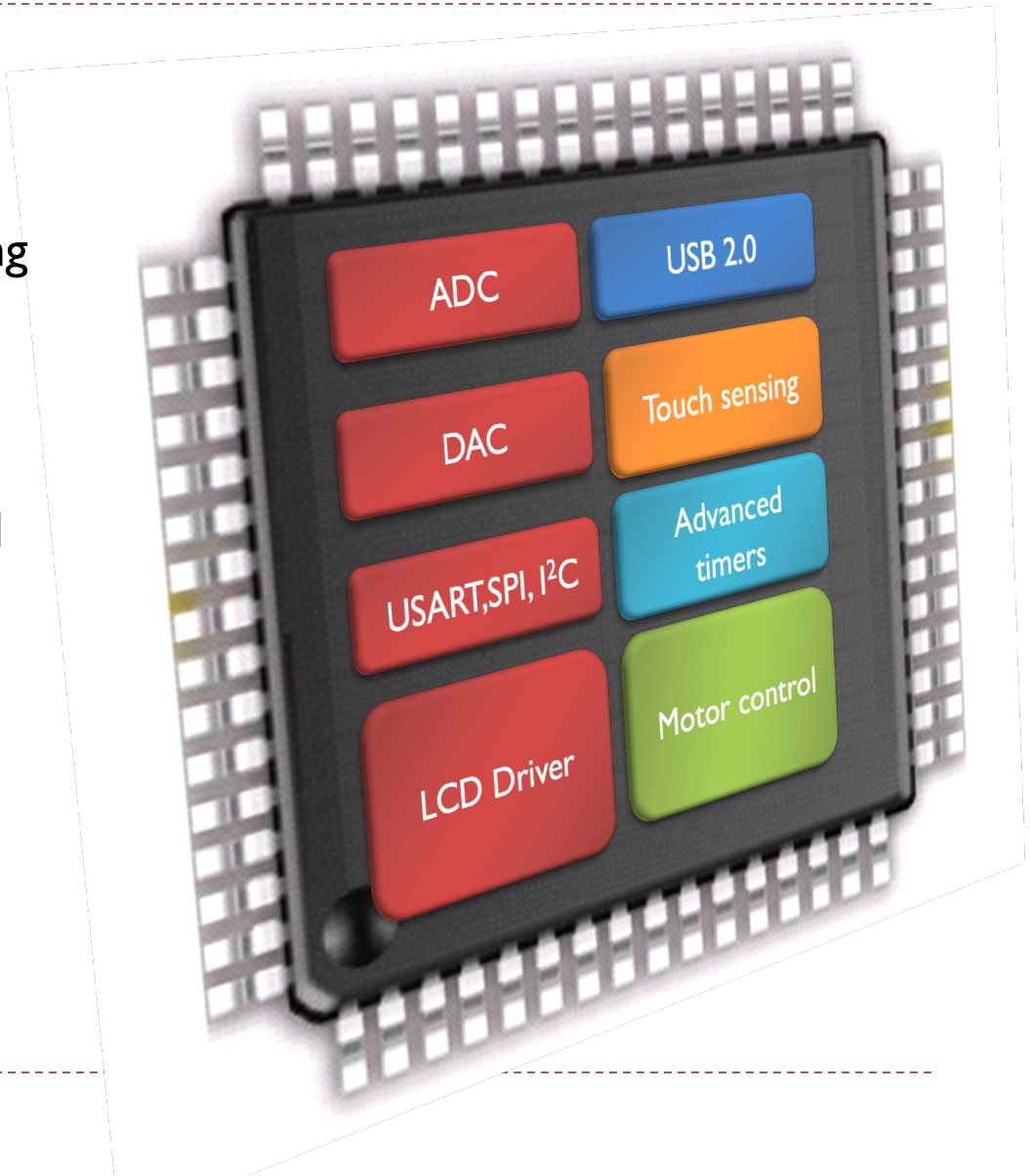
x = 1;



x = 6

Why ARM processor

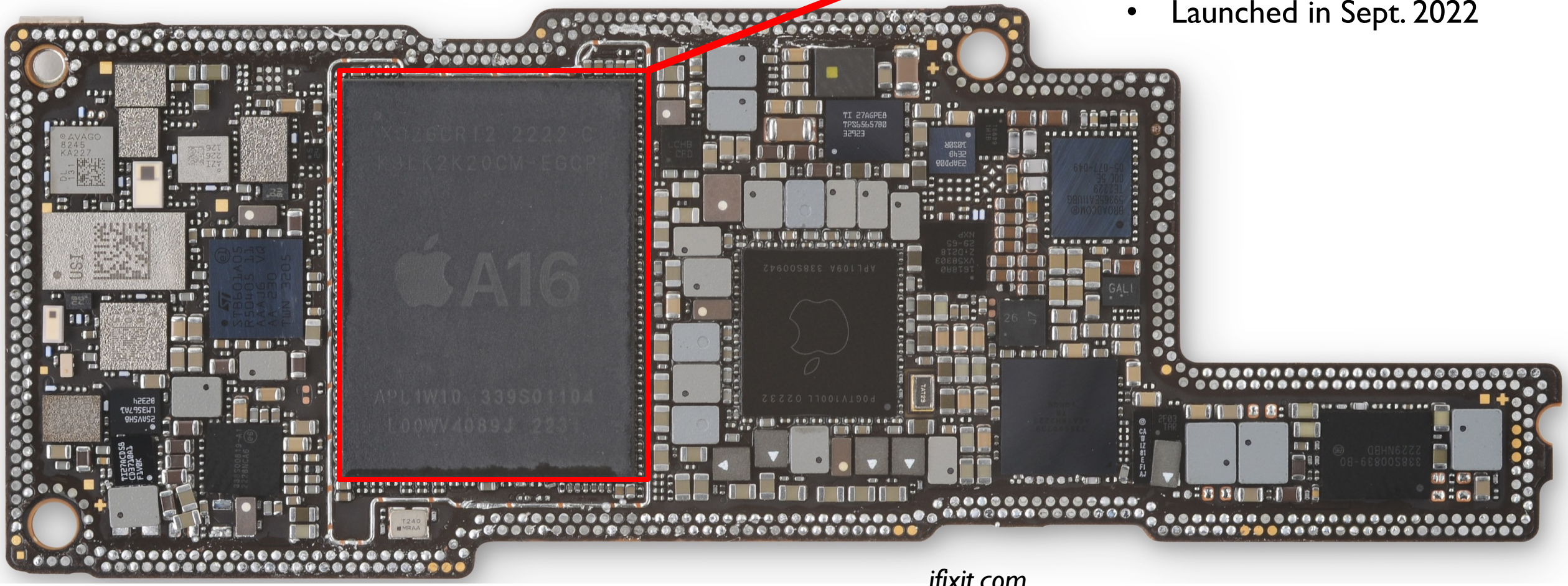
- ARM: Acron RISC Machine
- In 2010 alone, **6.1 billion** ARM-based processor, representing **95%** of smartphones, **35%** of digital televisions and set-top boxes and **10%** of mobile computers
- As of 2019, **150** billion ARM processors have been produced



iPhone 14

A16 Bionic:

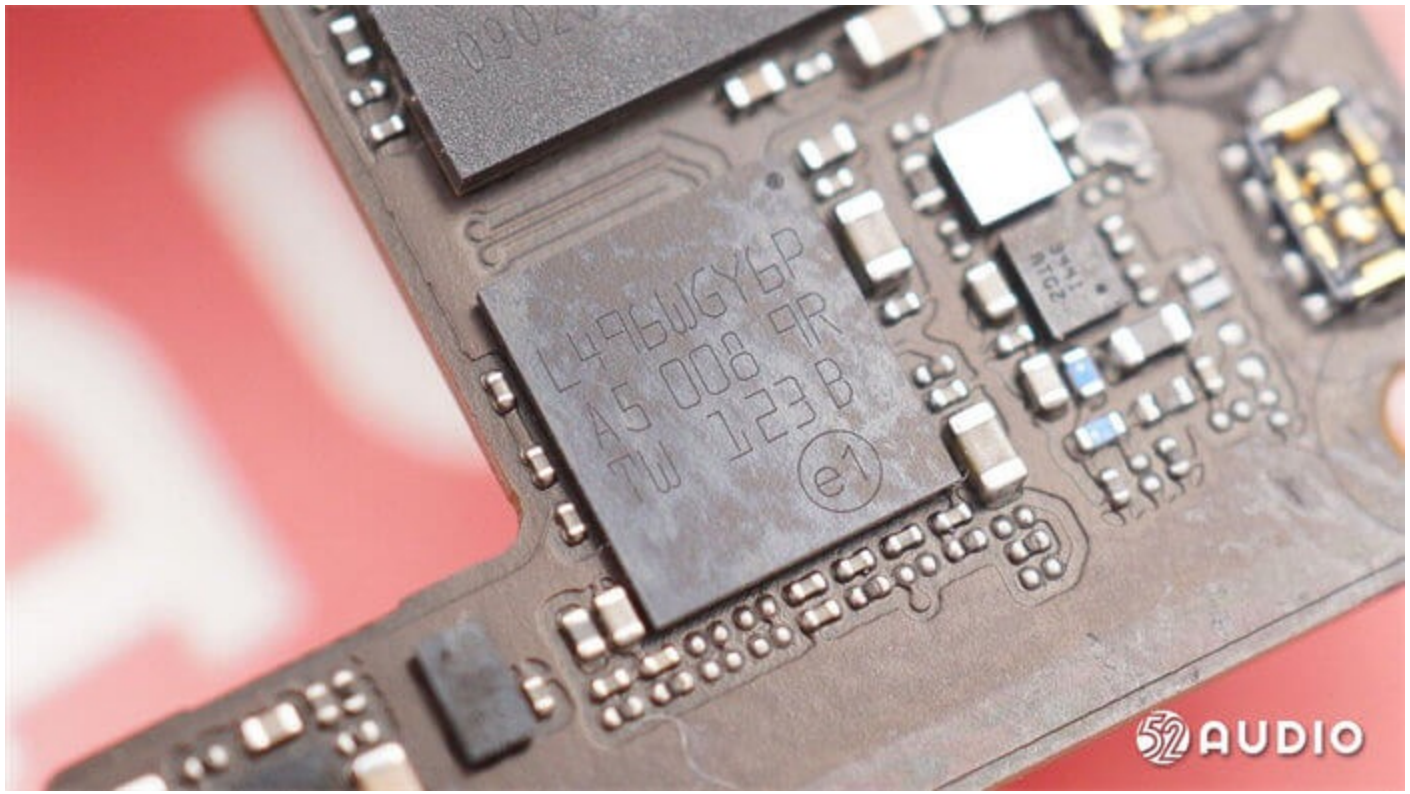
- 64-bit system on chip (SoC)
- Arm AArch64 Instruction Set
- Launched in Sept. 2022



ifixit.com

Apple AirPods 3

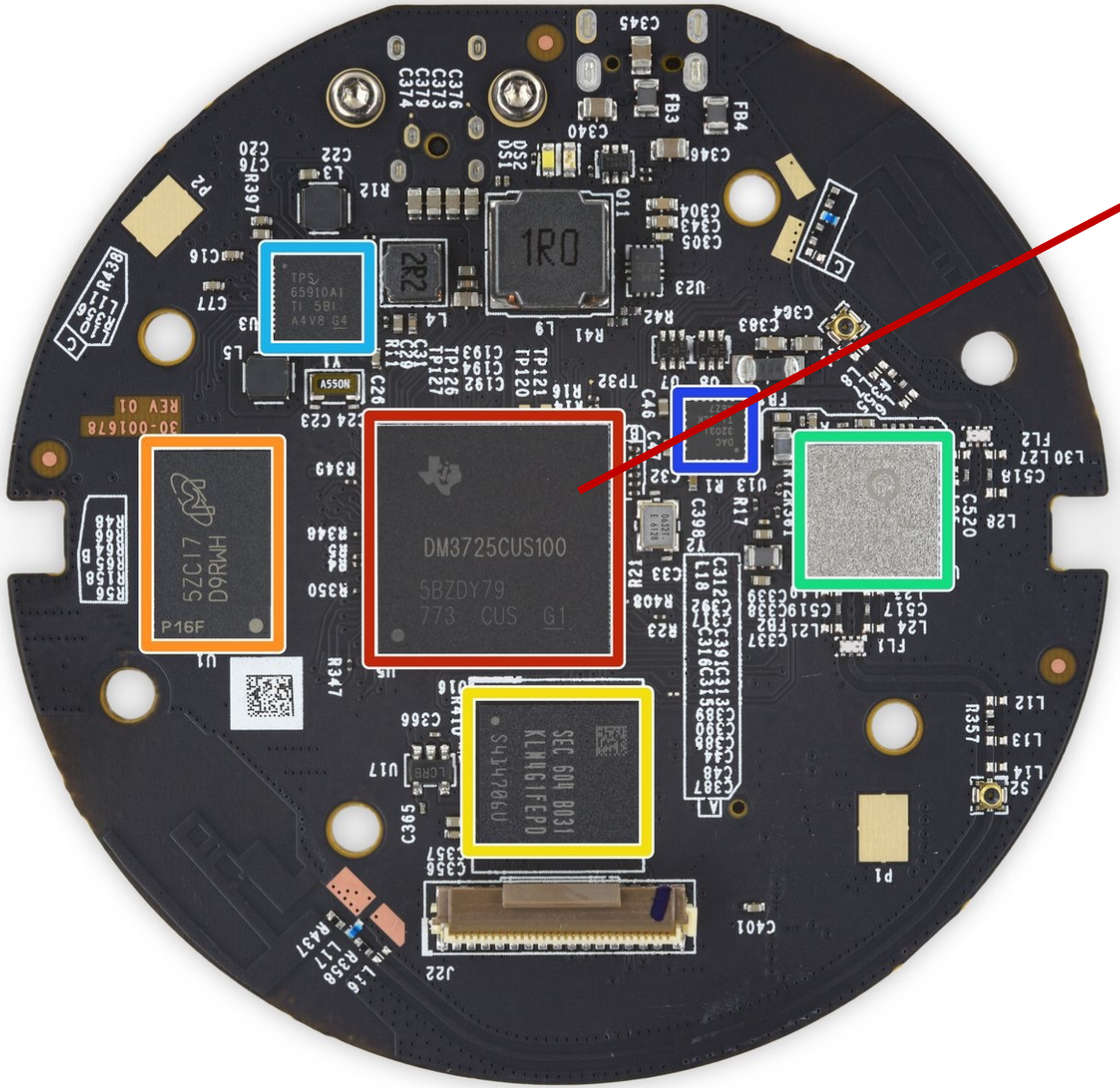




- On Charge Station:

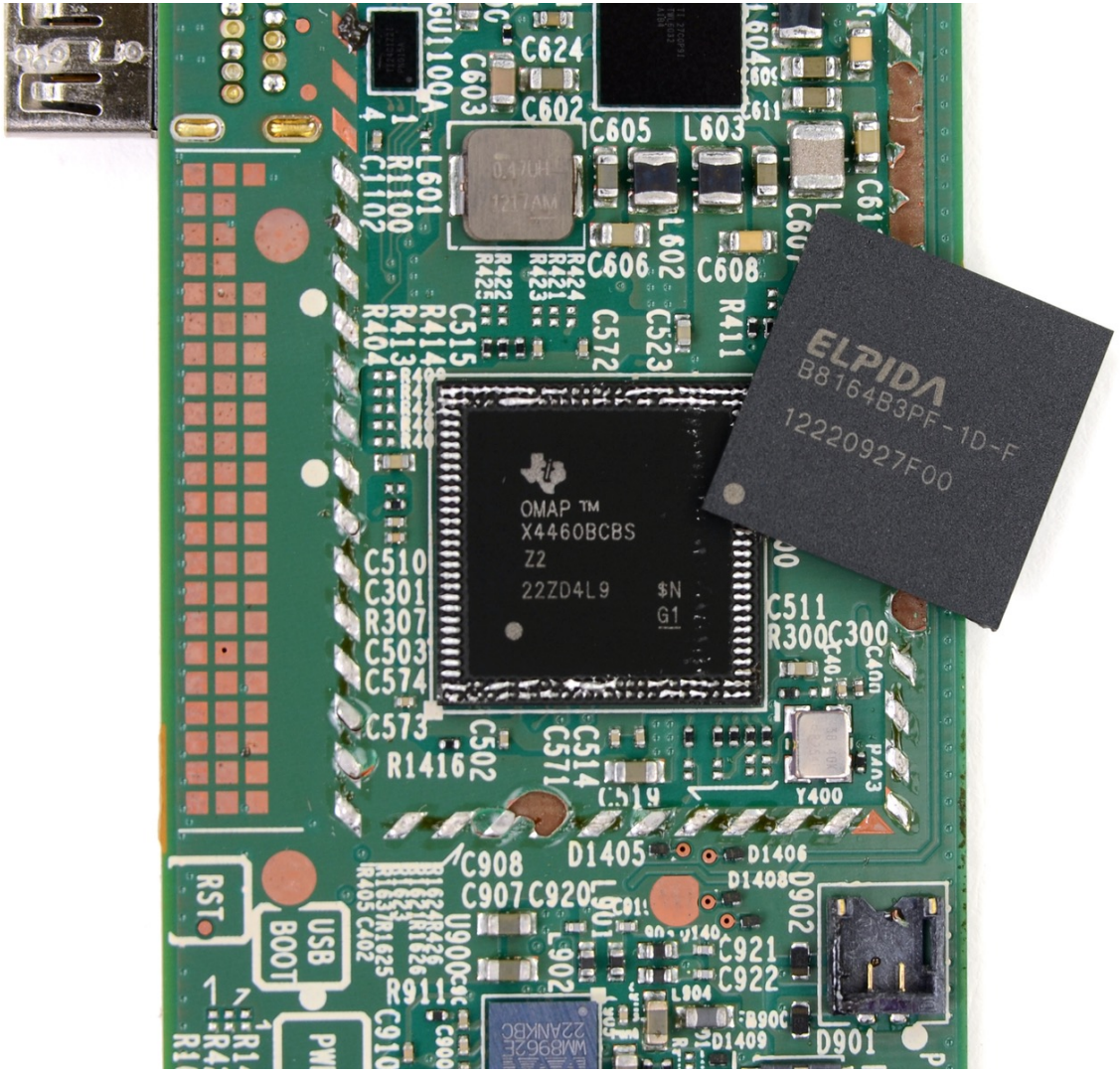
STM32L496: Arm Cortex-M4 32-bit MCU+FPU

Amazon Echo (Alexa)



- Texas Instruments DM3725 Digital Media Processor
  - ARM Cortex-A8

Kindle HD Fire

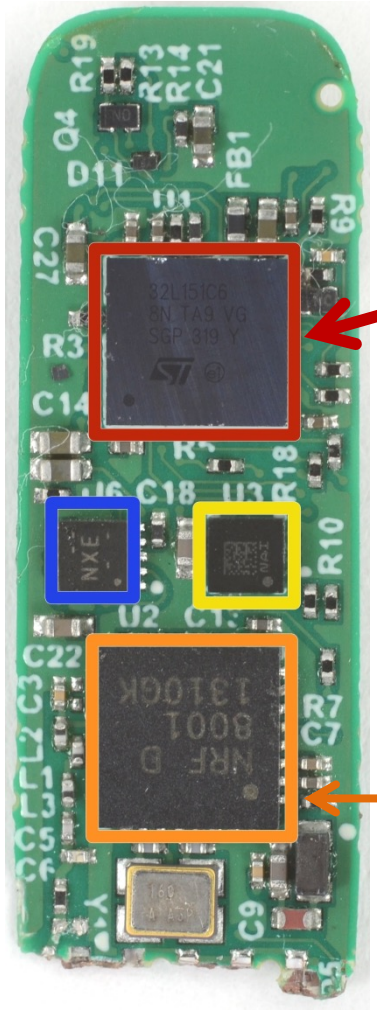


Texas Instruments **OMAP 4460** dual-core processor

http://www.ifixit.com

Fitbit Flex Teardown



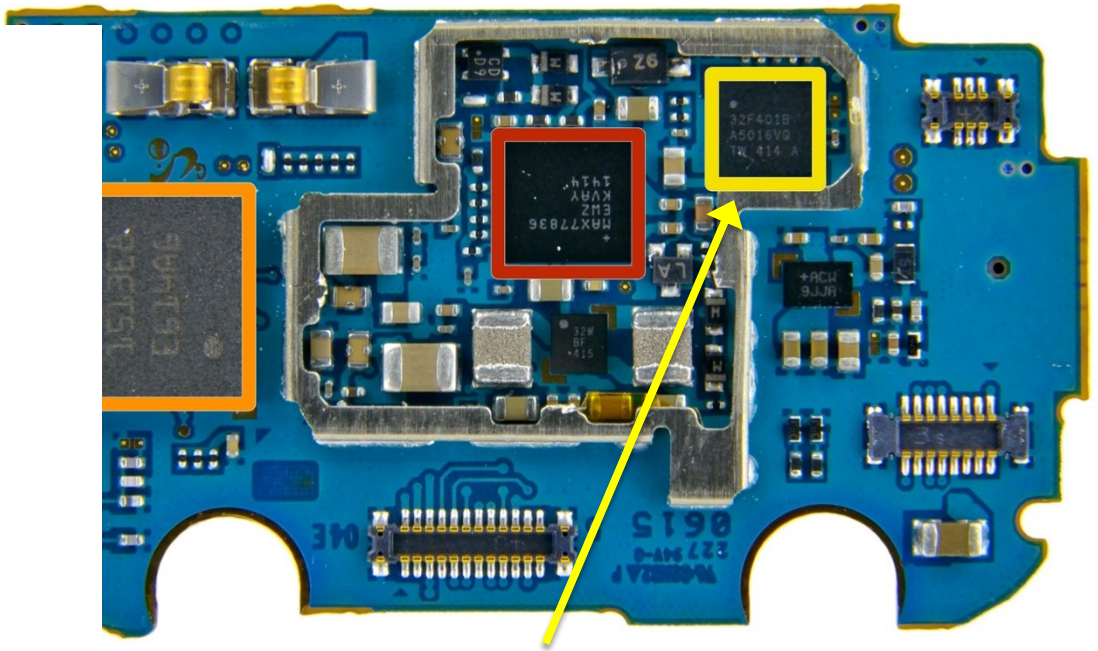


STMicroelectronics 32L151C6 Ultra Low Power ARM Cortex M3 Microcontroller

Nordic Semiconductor nRF8001 Bluetooth Low Energy Connectivity IC

Samsung Galaxy Gear



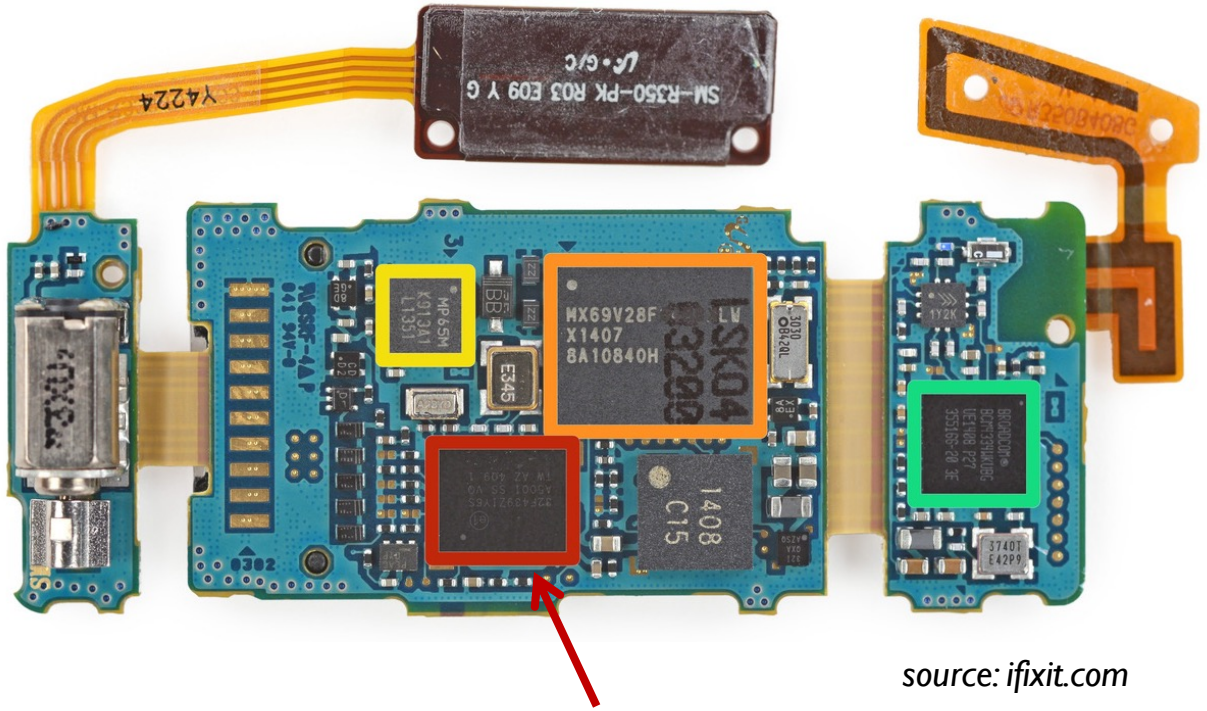


- STMicroelectronics STM32F401B **ARM-Cortex M4** MCU with 128KB Flash

source: ifixit.com

Samsung Gear Fit Fitness Tracker



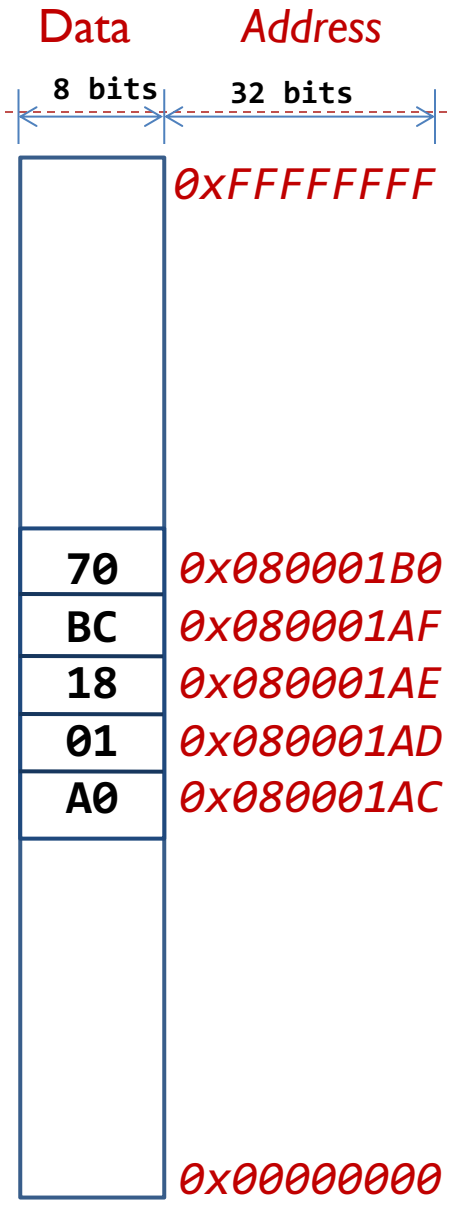


source: *ifixit.com*

- STMicroelectronics **STM32F4**39ZI 180 MHz, 32 bit **ARM Cortex-M4** CPU

Memory

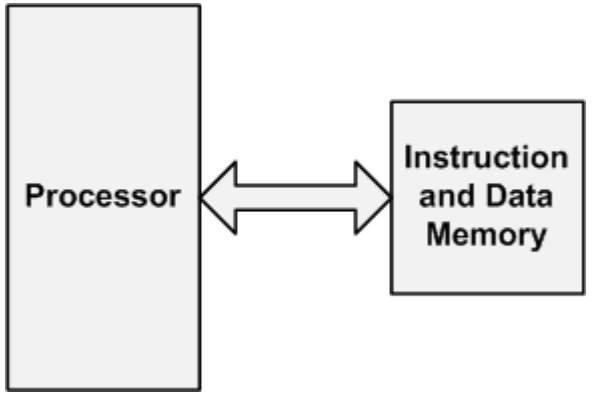
- Memory is arranged as a series of "locations"
  - Each location has a unique "address"
  - Each location holds a byte (***byte-addressable***)
  - e.g. the memory location at address `0x080001B0` contains the byte value `0x70`, i.e., 112
- The number of locations in memory is limited
  - e.g. 4 GB of RAM
  - 1 Gigabyte (GB) = 2^{30} bytes
  - 2^{32} locations ➔ 4,294,967,296 locations!
- Values stored at each location can represent either program data or program instructions
  - e.g. the value `0x70` might be the code used to tell the processor to add two values together



Computer Architecture

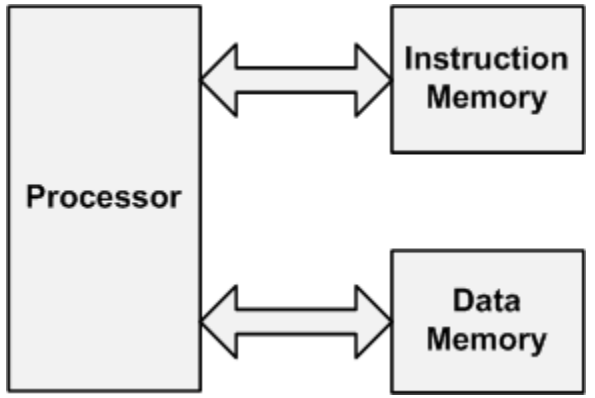
Von-Neumann

Instructions and data are stored in the same memory.



Harvard

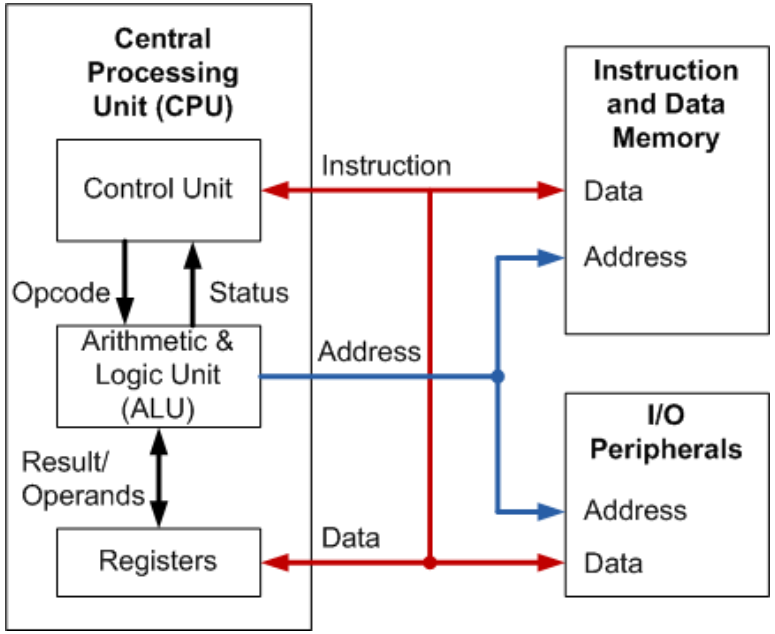
Data and instructions are stored into separate memories.



Computer Architecture

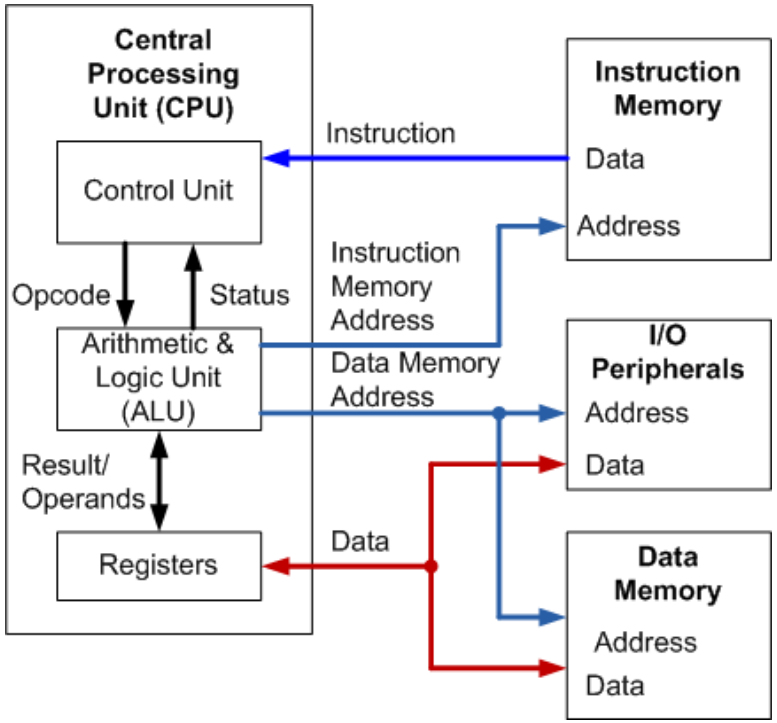
Von-Neumann

Instructions and data are stored in the same memory.



Harvard

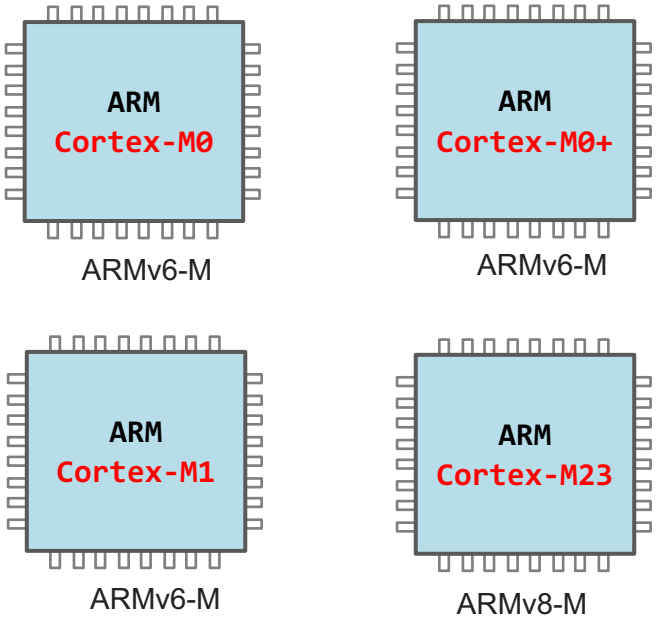
Data and instructions are stored into separate memories.



ARM Cortex-M Series Family

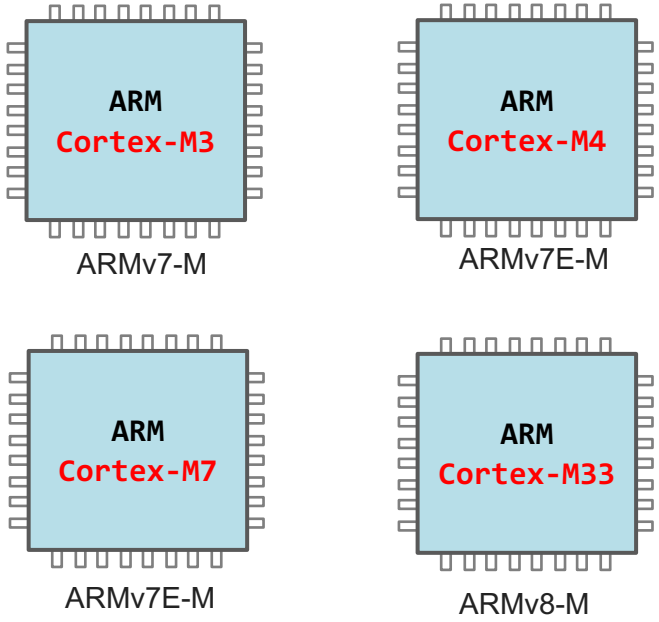
Von-Neumann

Instructions and data are stored in the same memory.



Harvard

Data and instructions are stored into separate memories.



Levels of Program Code

C Program

```
int main(void){
  int i;
  int total = 0;
  for (i = 0; i < 10; i++) {
    total += i;
  }
  while(1); // Dead loop
}
```

Compile →

Assembly Program

```
        MOVS r1, #0
        MOVS r0, #0
        B    check
loop    ADD  r1, r1, r0
        ADDS r0, r0, #1
check   CMP  r0, #10
        BLT  loop
self    B    self
```

Assemble →

Machine Program

```
0010000100000000
0010000000000000
1110000000000001
0100010000000001
0001110001000000
0010100000001010
1101110011111011
1011111100000000
1110011111111110
```

- **High-level language**
  - Level of abstraction closer to problem domain
  - Provides for productivity and portability
- **Assembly language**
  - Textual representation of instructions
  - Human-readable format instructions
- **Hardware representation**
  - Binary digits (bits)
  - Encoded instructions and data
  - Computer-readable format instructions

See a Program Runs

C Code

```
int main(void){
  int a = 0;
  int b = 1;
  int c;
  c = a + b;
  return 0;
}
```

compiler

Assembly Code

```
MOVS r1, #0x00 ; int a = 0
MOVS r2, #0x01 ; int b = 1
ADDS r3, r1, r2; c = a + b
MOVS r0, 0x00  ; set return value
BX lr          ; return
```

assembler

Machine Code

```
0010000100000000
0010001000000001
0001100010001011
0010000000000000
0100011101110000
```

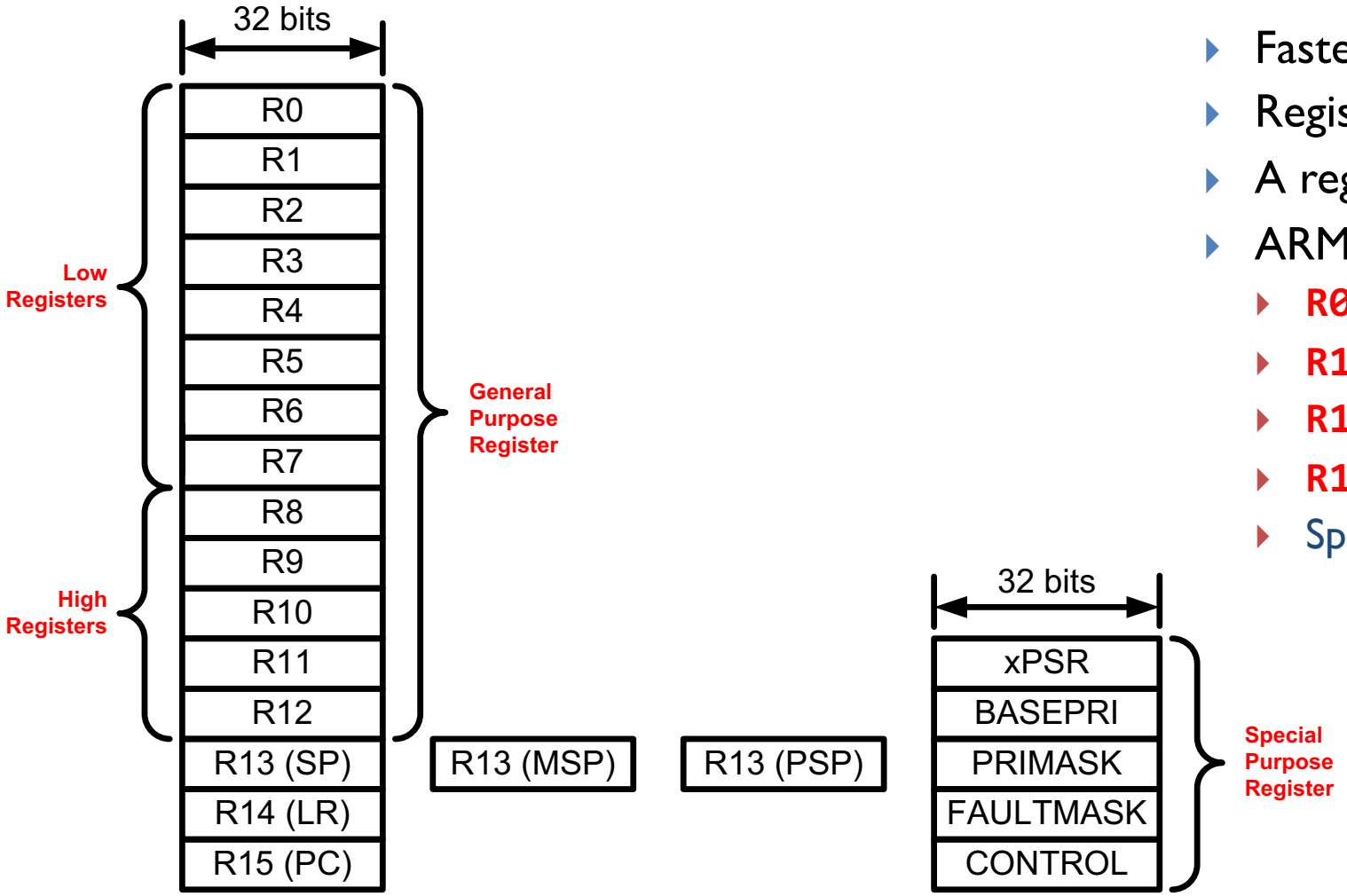
In Binary

```
2100
2201
188B
2000
4770
```

In Hex

```
; MOVS r1, #0x00
; MOVS r2, #0x01
; ADDS r3, r1, r2
; MOVS r0, #0x00
; BX lr
```

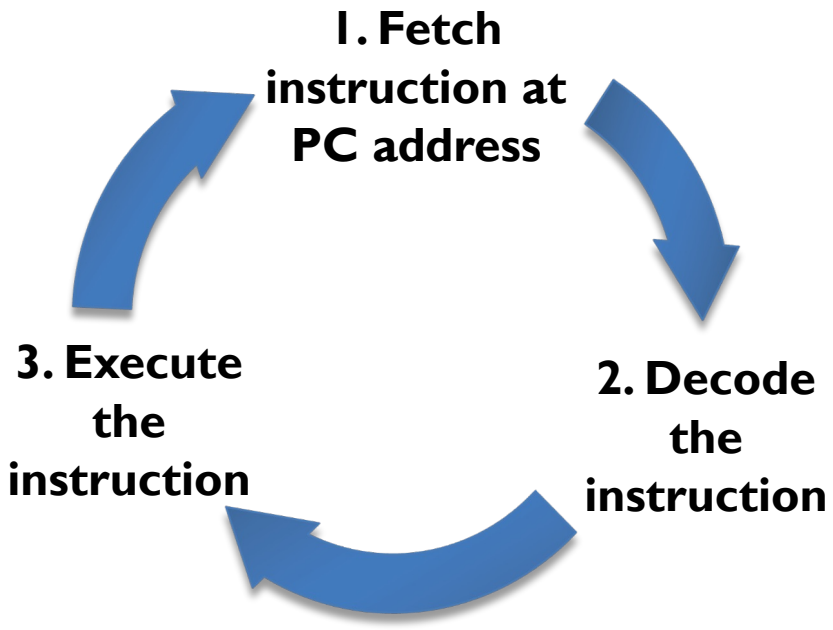
Processor Registers

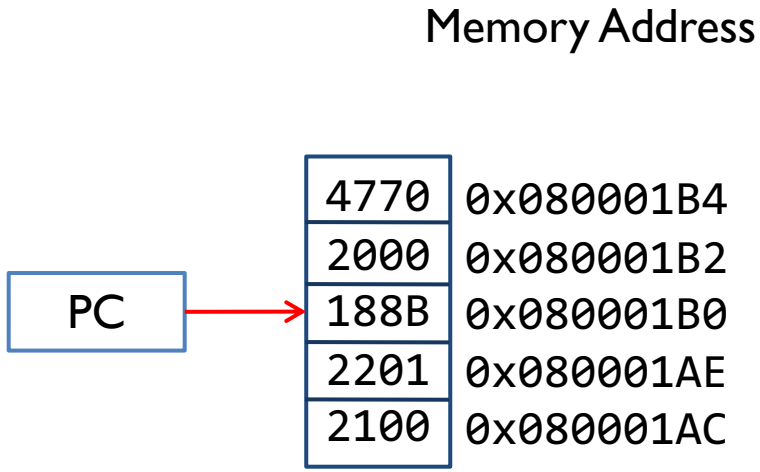


- Fastest way to read and write
- Registers are within the processor chip
- A register stores 32-bit value
- ARM Cortex-M has
  - **R0-R12**: 13 general-purpose registers
  - **R13**: Stack pointer (Shadow of MSP or PSP)
  - **R14**: Link register (LR)
  - **R15**: Program counter (PC)
  - Special registers (xPSR, BASEPRI, PRIMASK, etc)

Program Execution

- **Program Counter (PC)** is a register that holds the memory address of the next instruction to be fetched from the memory.

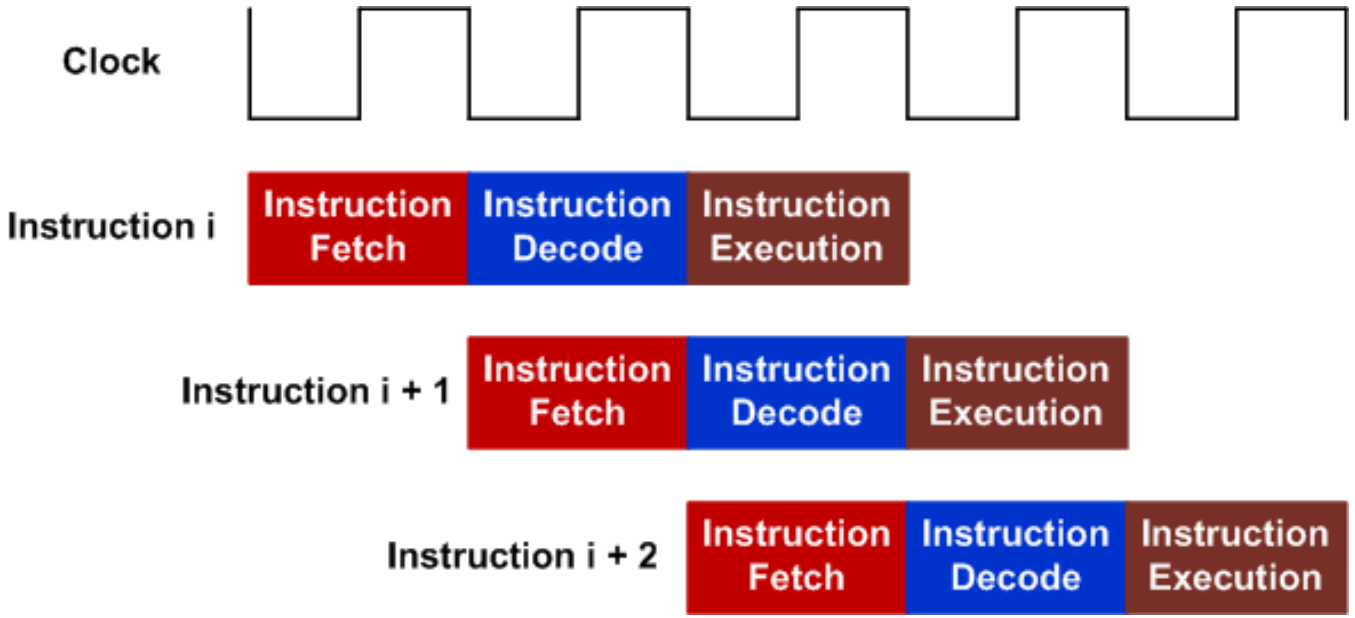




PC = 0x080001B0
Instruction = 188B or
2000188B or 8B180020

Three-state pipeline: Fetch, Decode, Execution

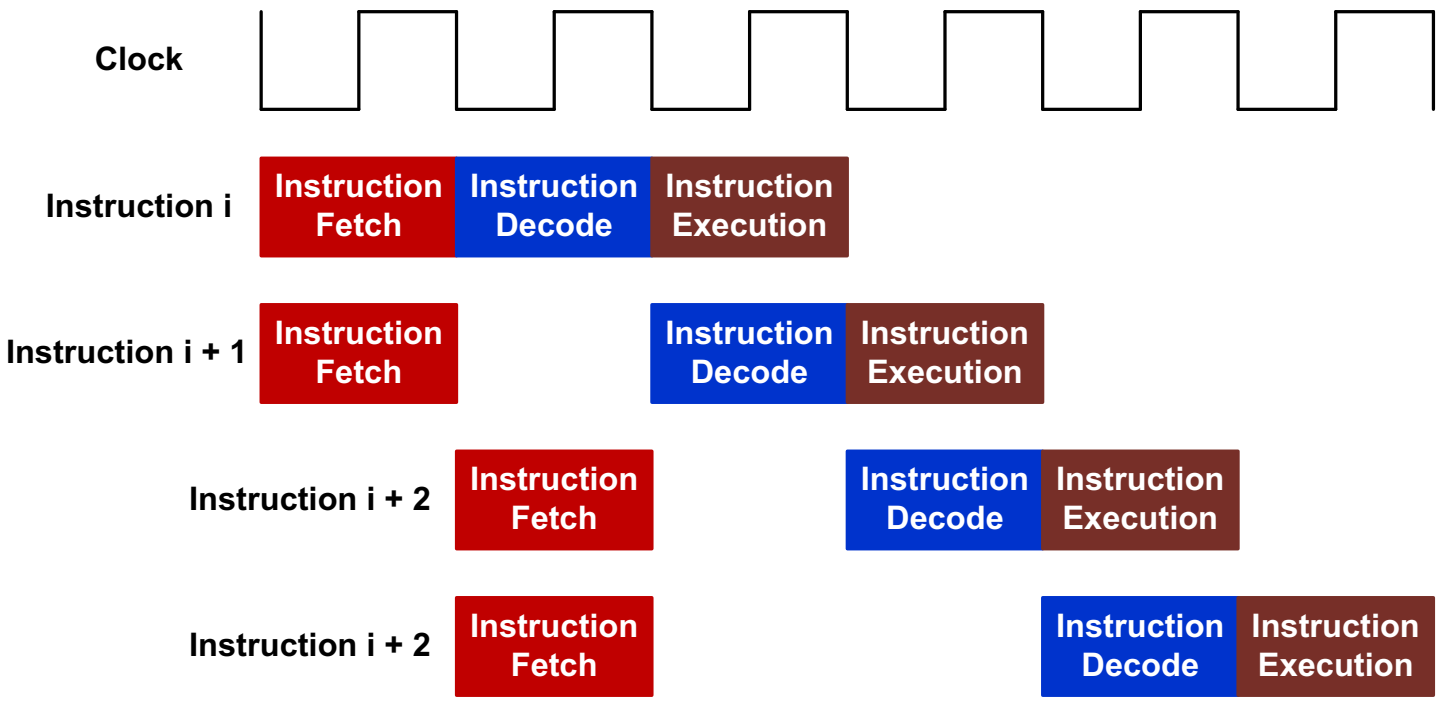
- **Pipelining** allows hardware resources to be fully utilized
- One 32-bit instruction or two 16-bit instructions can be fetched.



Pipeline of 32-bit instructions

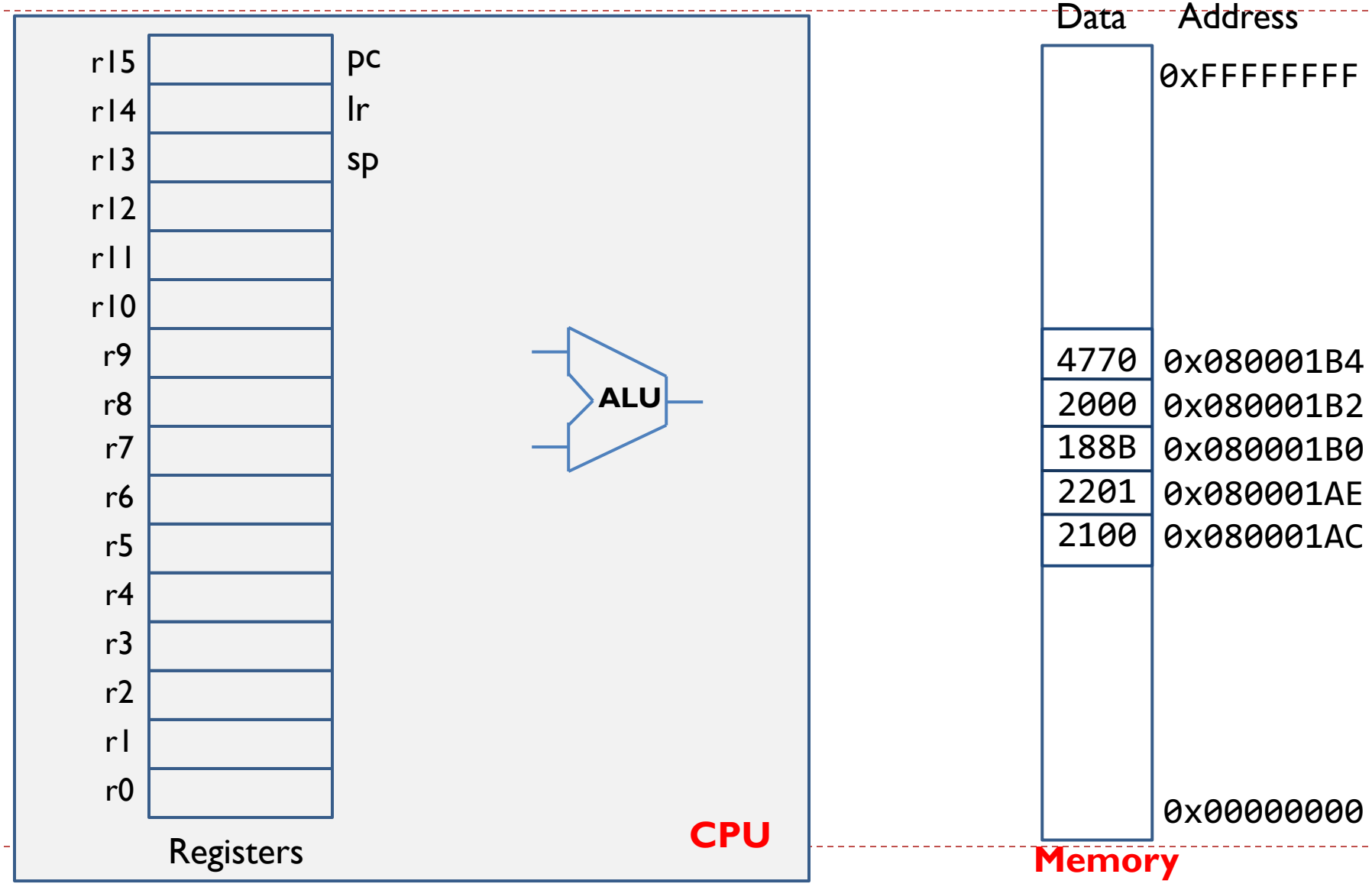
Three-state pipeline: Fetch, Decode, Execution

- **Pipelining** allows hardware resources to be fully utilized
- One 32-bit instruction or two 16-bit instructions can be fetched.

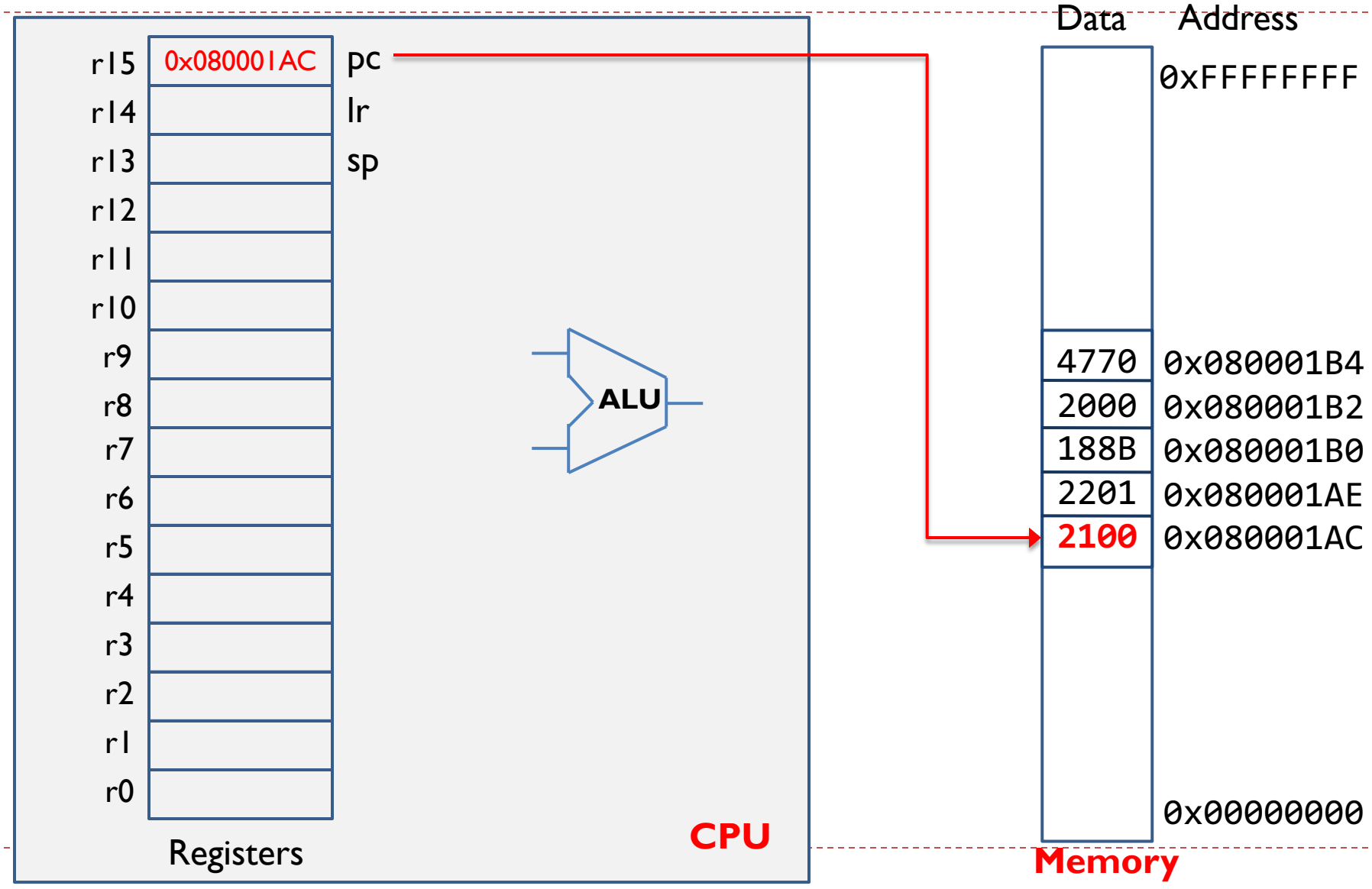


Pipeline of 16-bit instructions

Machine codes are stored in memory

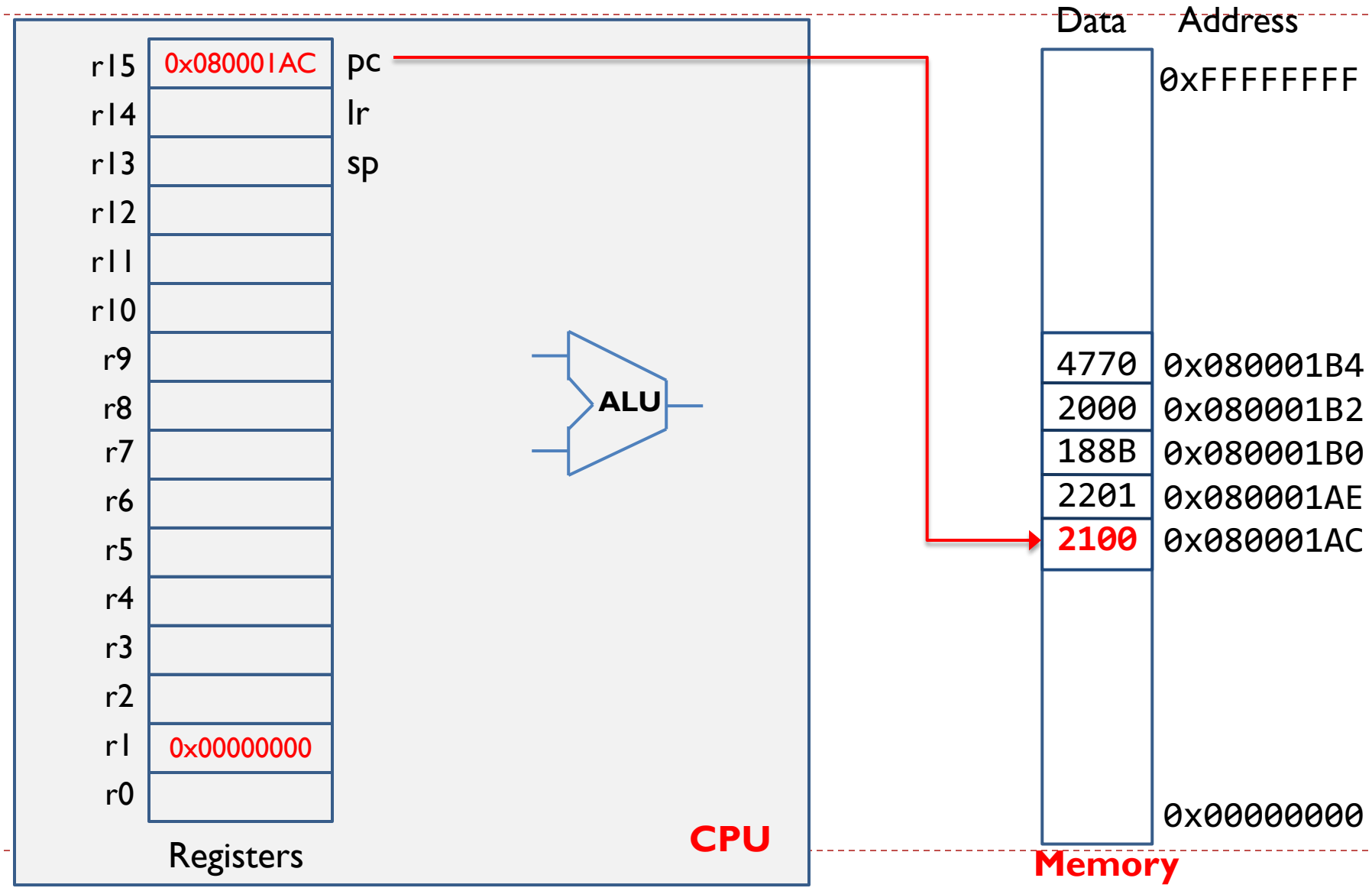


Fetch Instruction: pc = 0x08001AC
Decode Instruction: 2100 = MOVS r1, #0x00



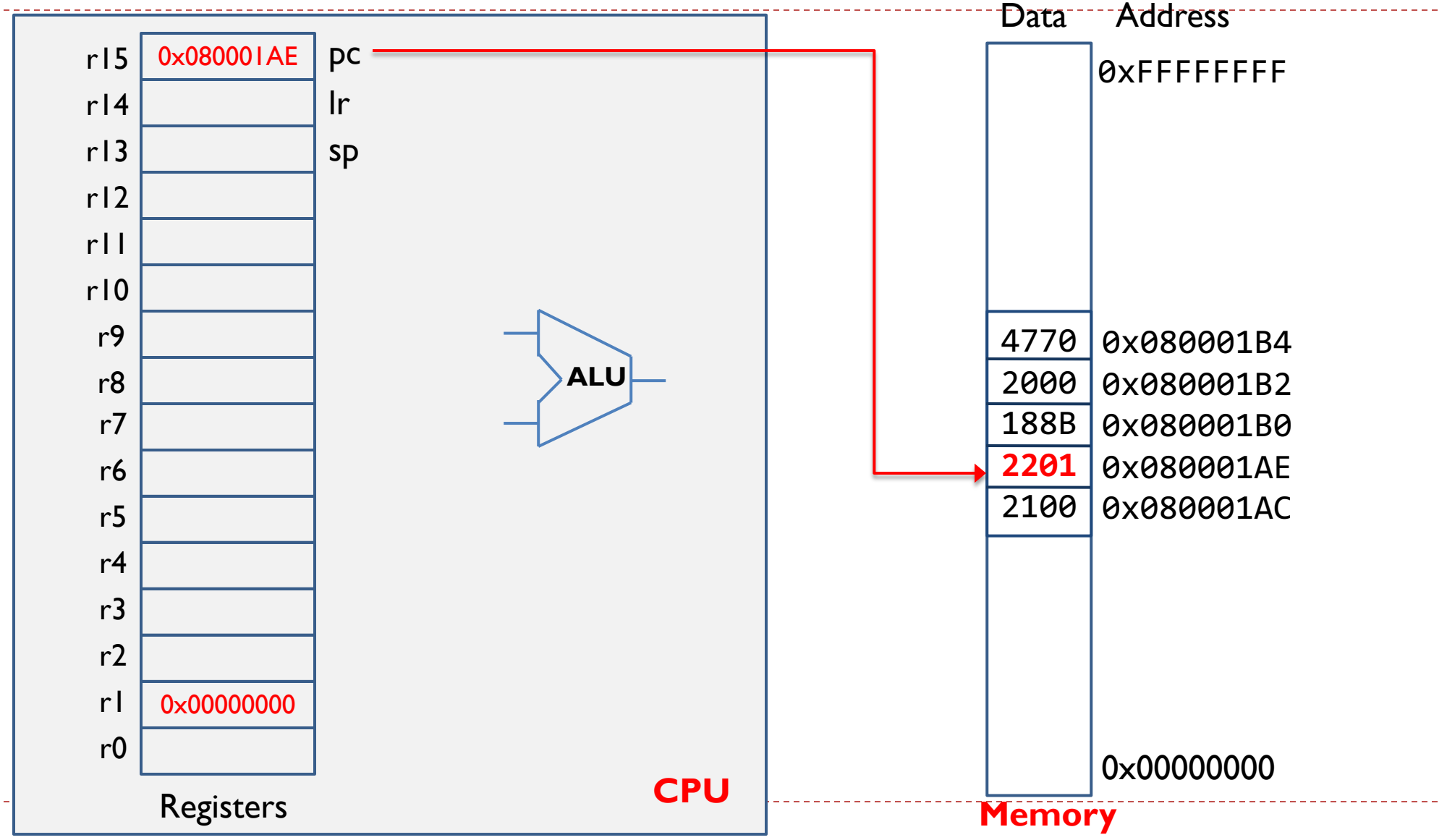
Execute Instruction:

MOVS r1, #0x00

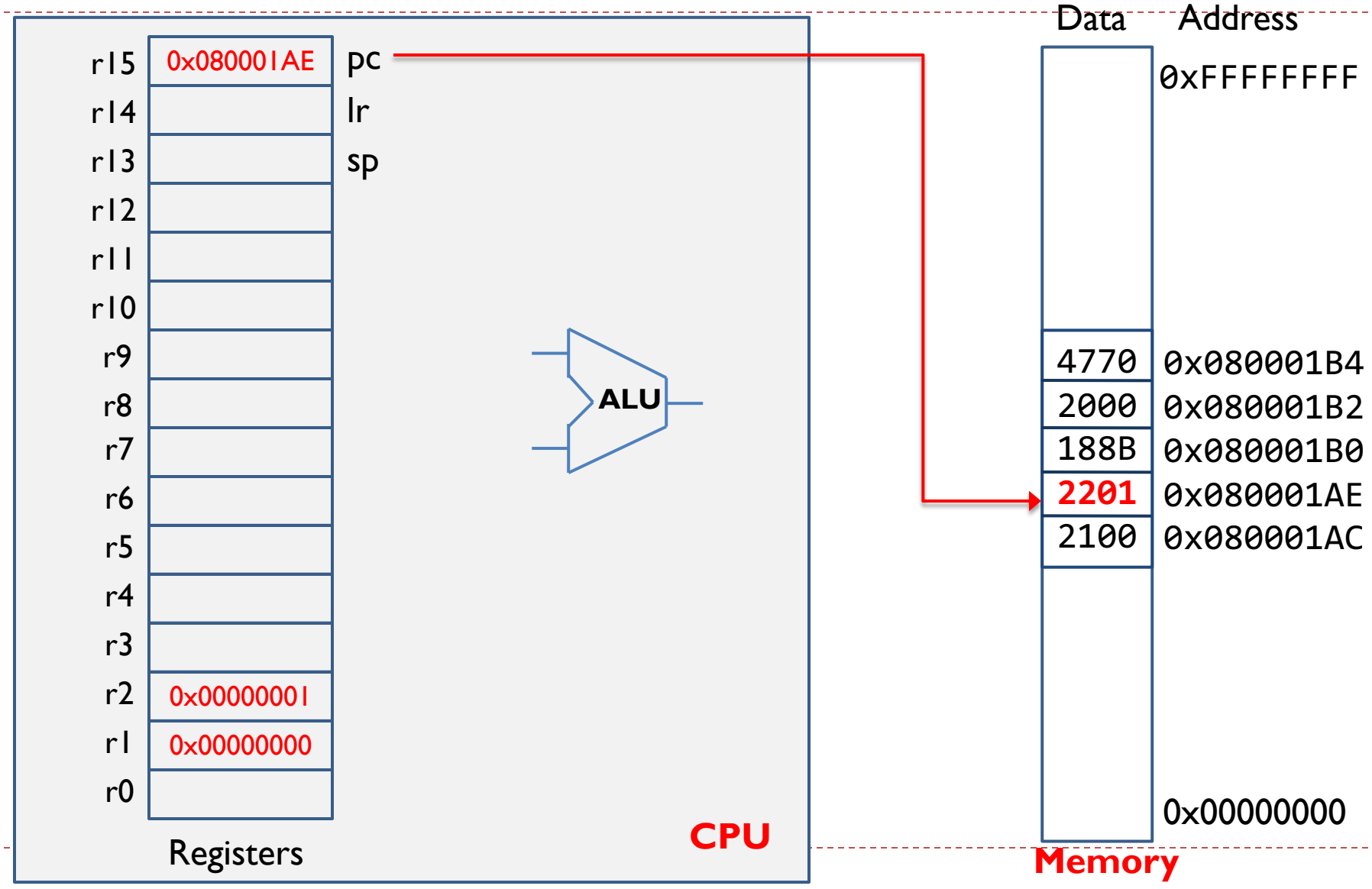


Fetch Next Instruction: pc = pc + 2

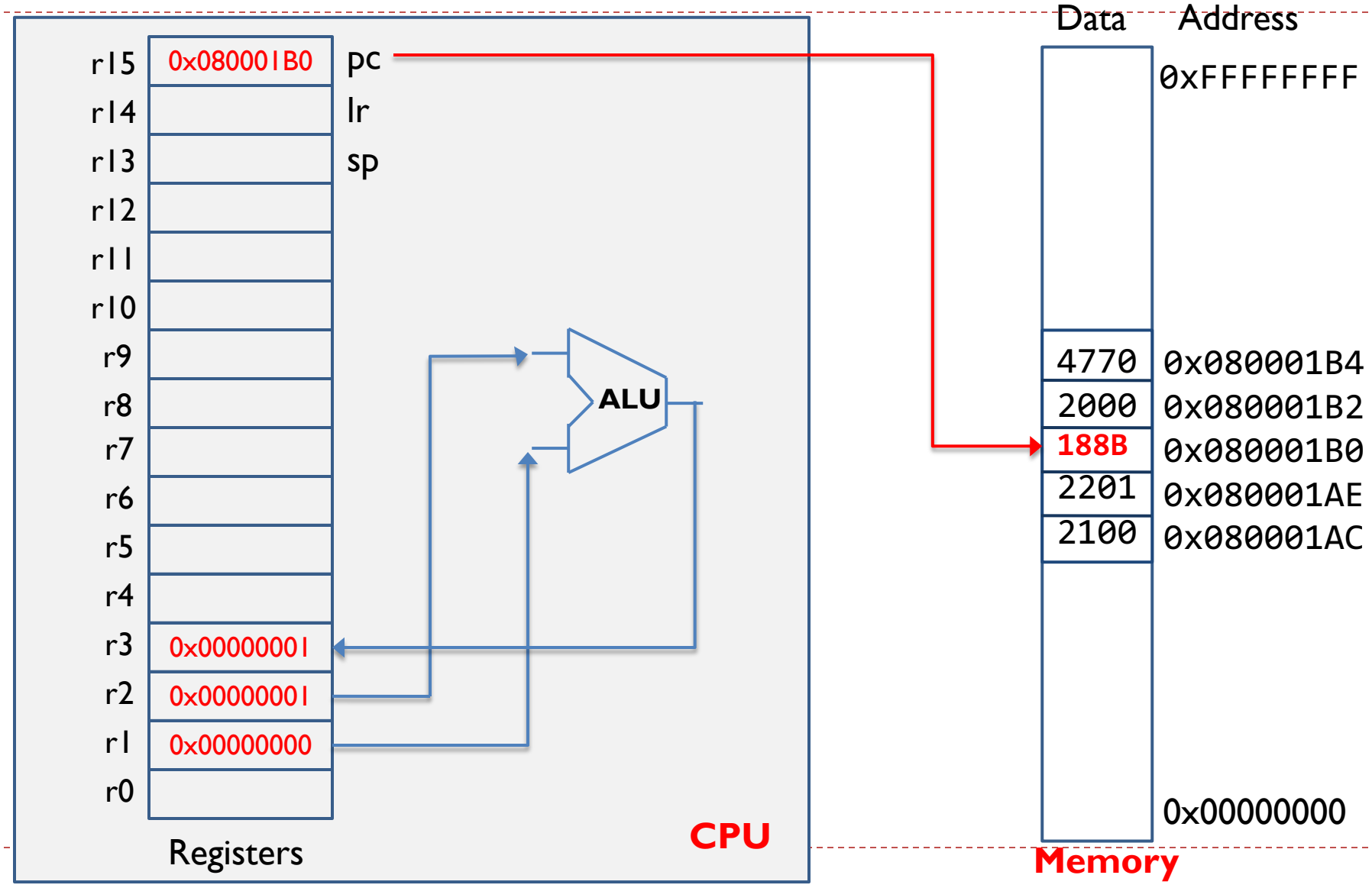
- Thumb-2 consists of a mix of 16- & 32-bit instructions
- In reality, we always fetch 4 bytes from the instruction memory (either one 32-bit instruction or two 16-bit instructions)
- To simplify the demo, we assume we only fetch 2 bytes from the instruction memory in this example.



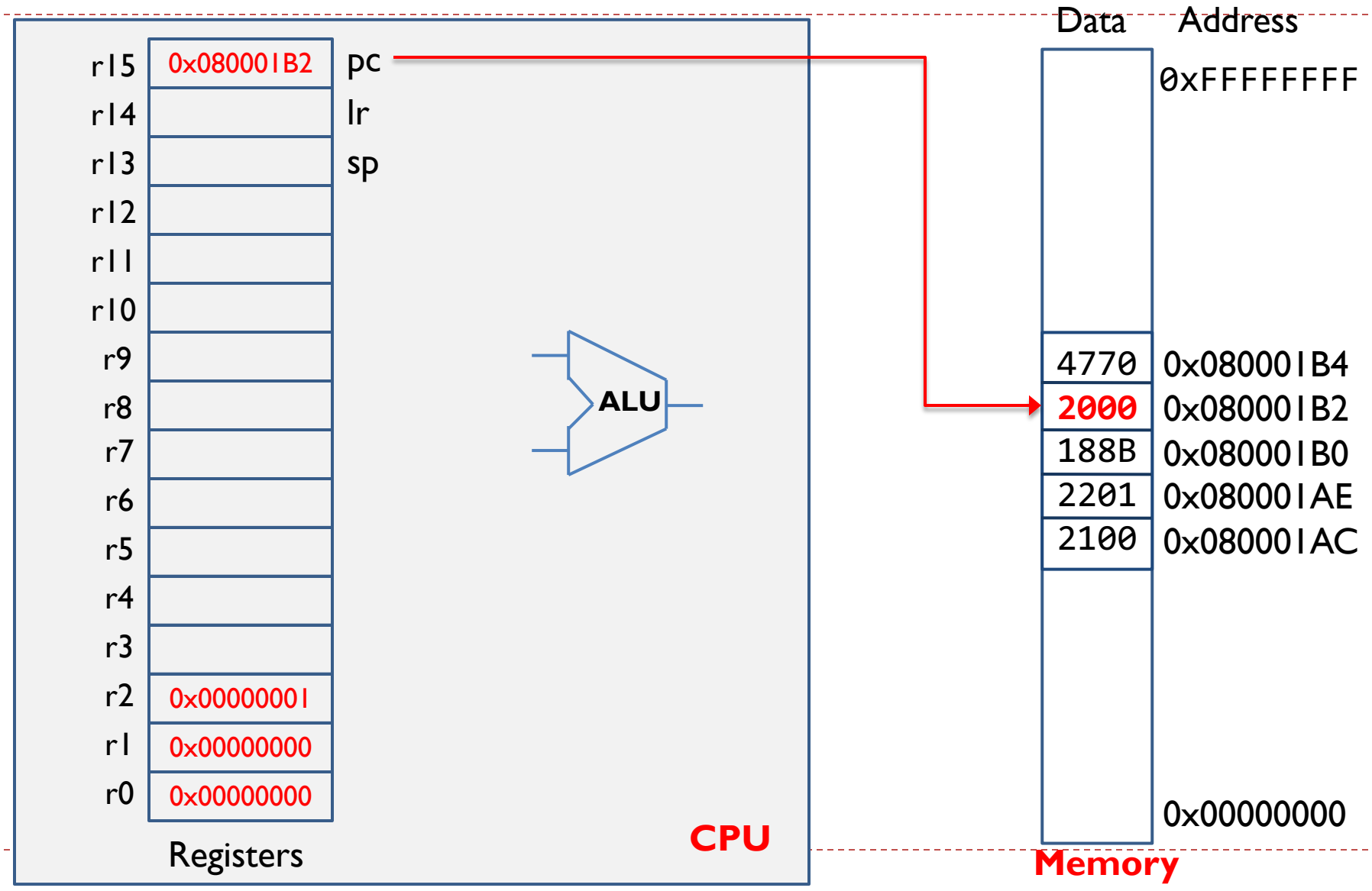
Fetch Next Instruction: pc = pc + 2
Decode & Execute: 2201 = MOVS r2, #0x01



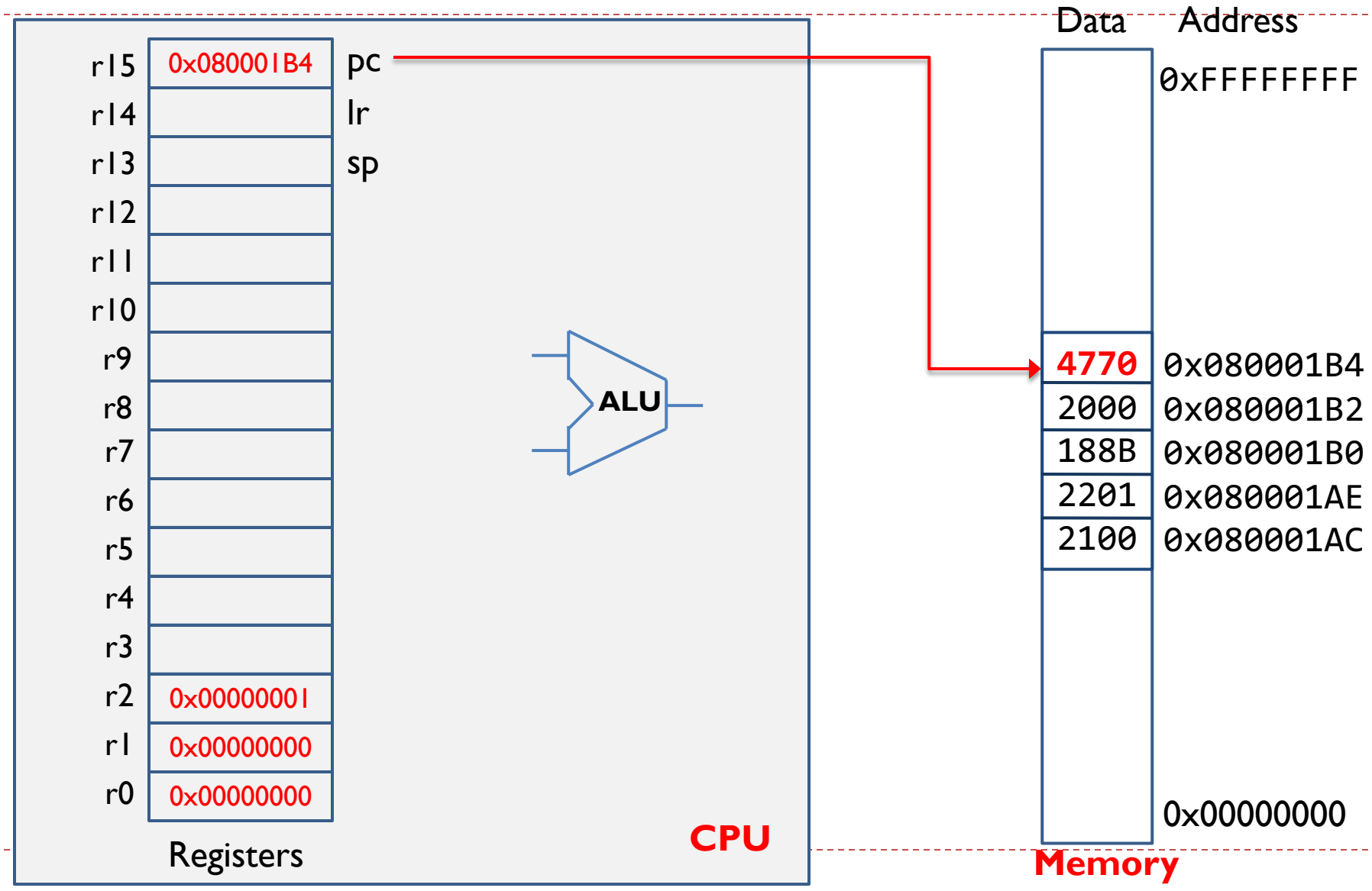
Fetch Next Instruction: pc = pc + 2
Decode & Execute: 188B = ADDS r3, r1, r2



Fetch Next Instruction: pc = pc + 2
Decode & Execute: 2000 = MOVS r0, #0x00



Fetch Next Instruction: pc = pc + 2
Decode & Decode: 4770 = BX lr



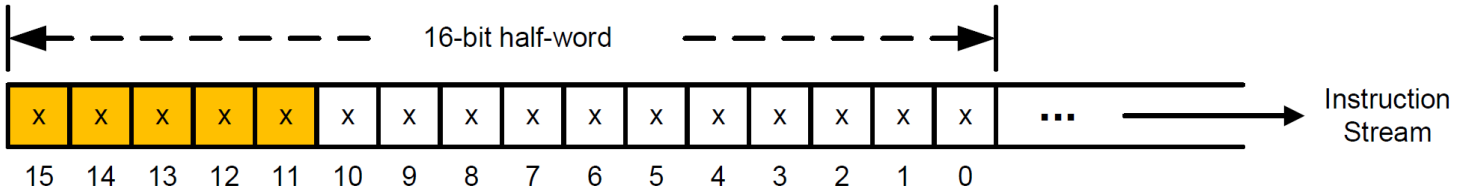
Realities

- In the previous example,
  - PC is incremented by 2

Well, I lied!

Realities

- PC is always incremented by **4**.
  - Each time, 4 bytes are fetched from the instruction memory
  - It is either two 16-bit instructions or one 32-bit instruction



If bit [15-11] = `11101`, `11110`, or `11111`, then, it is the first half-word of a 32-bit instruction.

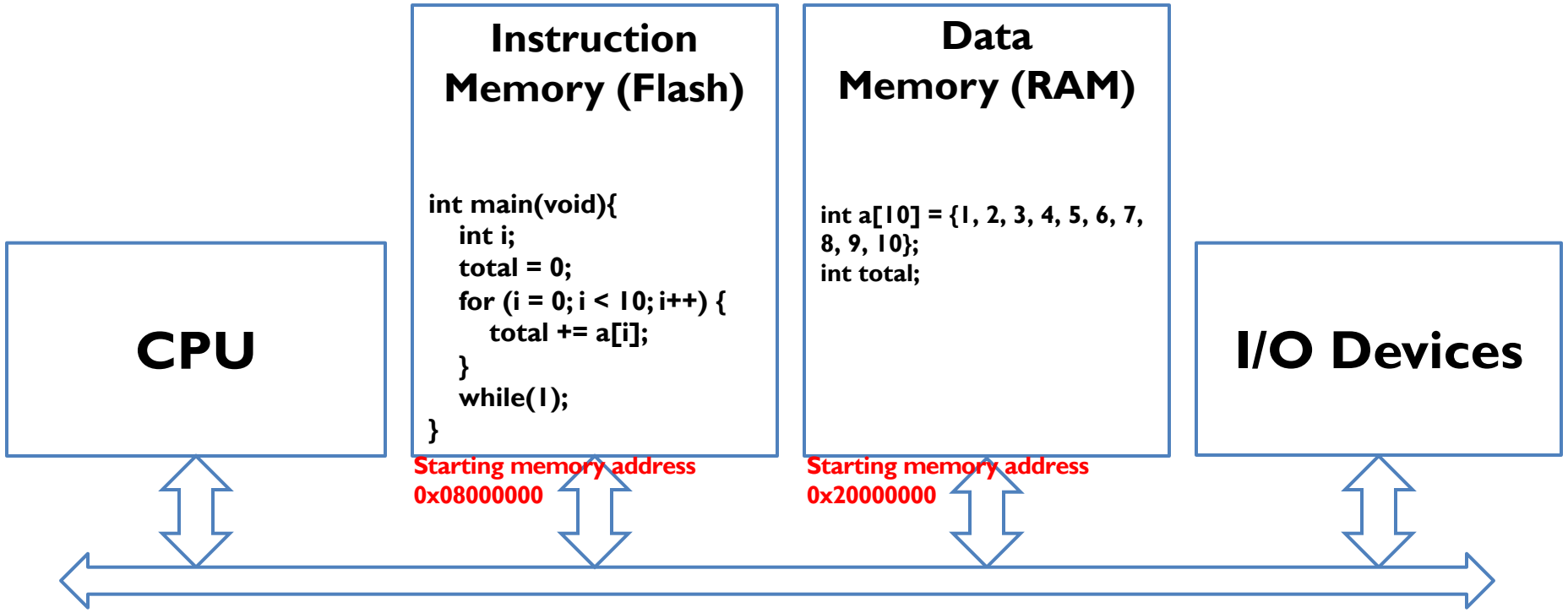
Otherwise, it is a 16-bit instruction.

Example:
Calculate the Sum of an Array

```
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int total;

int main(void){
  int i;
  total = 0;
  for (i = 0; i < 10; i++) {
    total += a[i];
  }
  while(1);
}
```

Example:
Calculate the Sum of an Array



Example:
Calculate the Sum of an Array

Instruction Memory (Flash)

```
int main(void){
    int i;
    total = 0;
    for (i = 0; i < 10; i++) {
        total += a[i];
    }
    while(1);
}
```

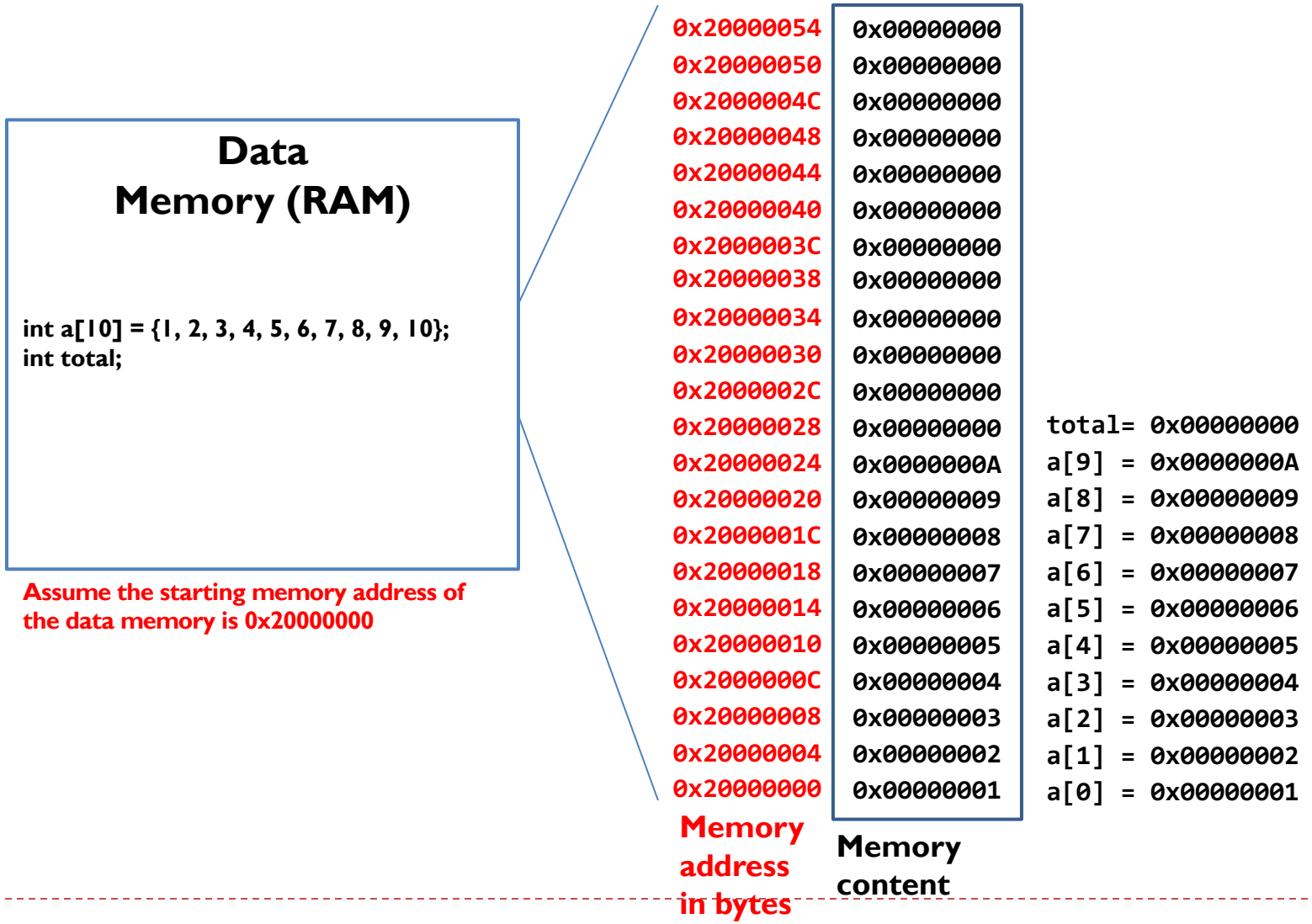
Starting memory address
0x08000000

```
0010 0001 0000 0000
0100 1010 0000 1000
0110 0000 0001 0001
0010 0000 0000 0000
1110 0000 0000 1000
0100 1001 0000 0111
1111 1000 0101 0001
0001 0000 0010 0000
0100 1010 0000 0100
0110 1000 0001 0010
0100 0100 0001 0001
0100 1010 0000 0011
0110 0000 0001 0001
0001 1100 0100 0000
0010 1000 0000 1010
1101 1011 1111 0100
1011 1111 0000 0000
1110 0111 1111 1110
```



```
        MOVS  r1, #0x00
        LDR   r2, = total_addr
        STR   r1, [r2, #0x00]
        MOVS  r0, #0x00
        B     Check
Loop:   LDR   r1, = a_addr
        LDR   r1, [r1, r0, LSL #2]
        LDR   r2, = total_addr
        LDR   r2, [r2, #0x00]
        ADD   r1, r1, r2
        LDR   r2, = total_addr
        STR   r1, [r2,#0x00]
        ADDS  r0, r0, #1
Check:  CMP   r0, #0x0A
        BLT   Loop
        NOP
Self:   B     Self
```

Example:
Calculate the Sum of an Array



Loading Code and Data into Memory

```
int counter;

int a[5] = {1, 2, 3, 4, 5};

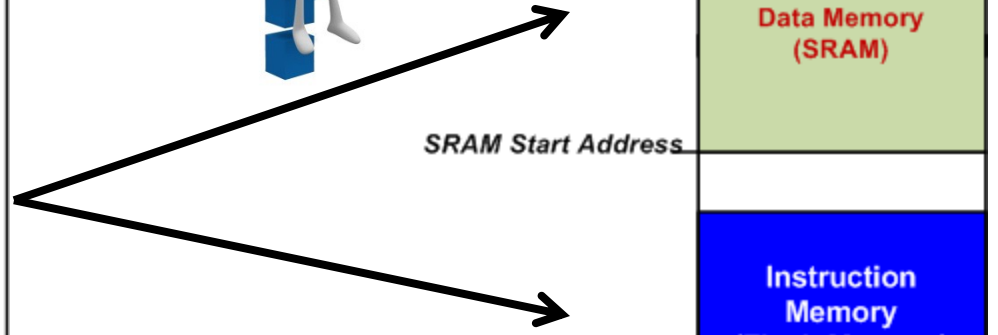
int main(void){
   int i;

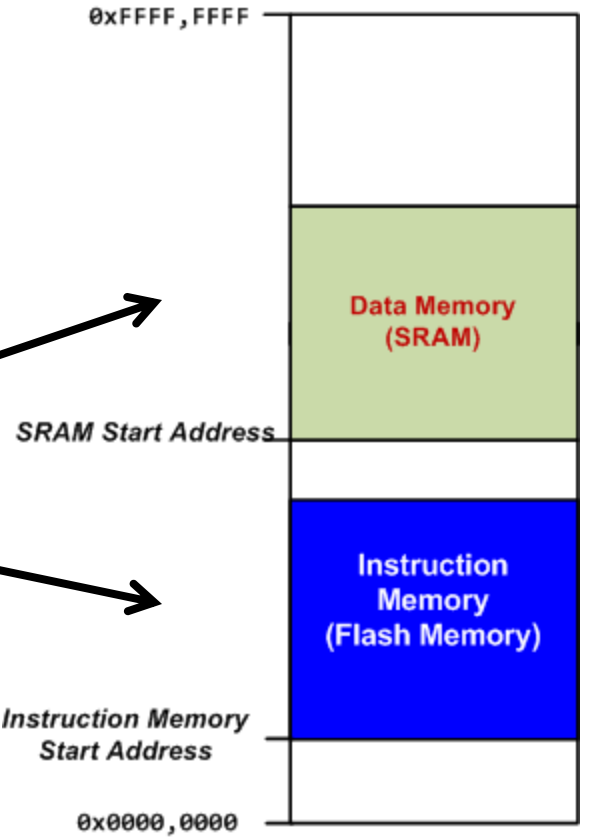
   int b[5];

   counter = 0;
   for (i = 0; i < 5; i++){
      b[i] = a[i];
      counter++
   }
   while(1);
}
```

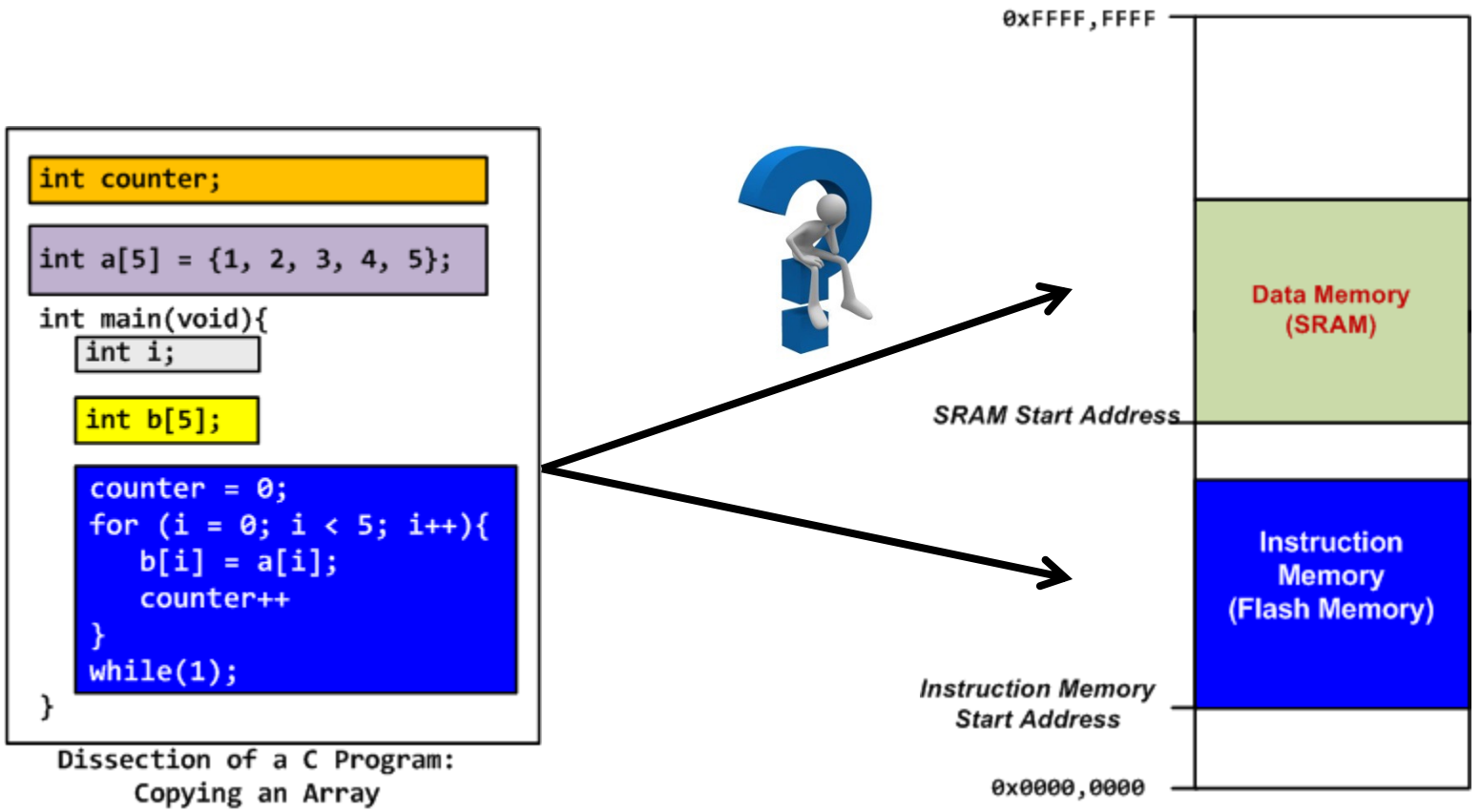
C Program:
Copying an Array



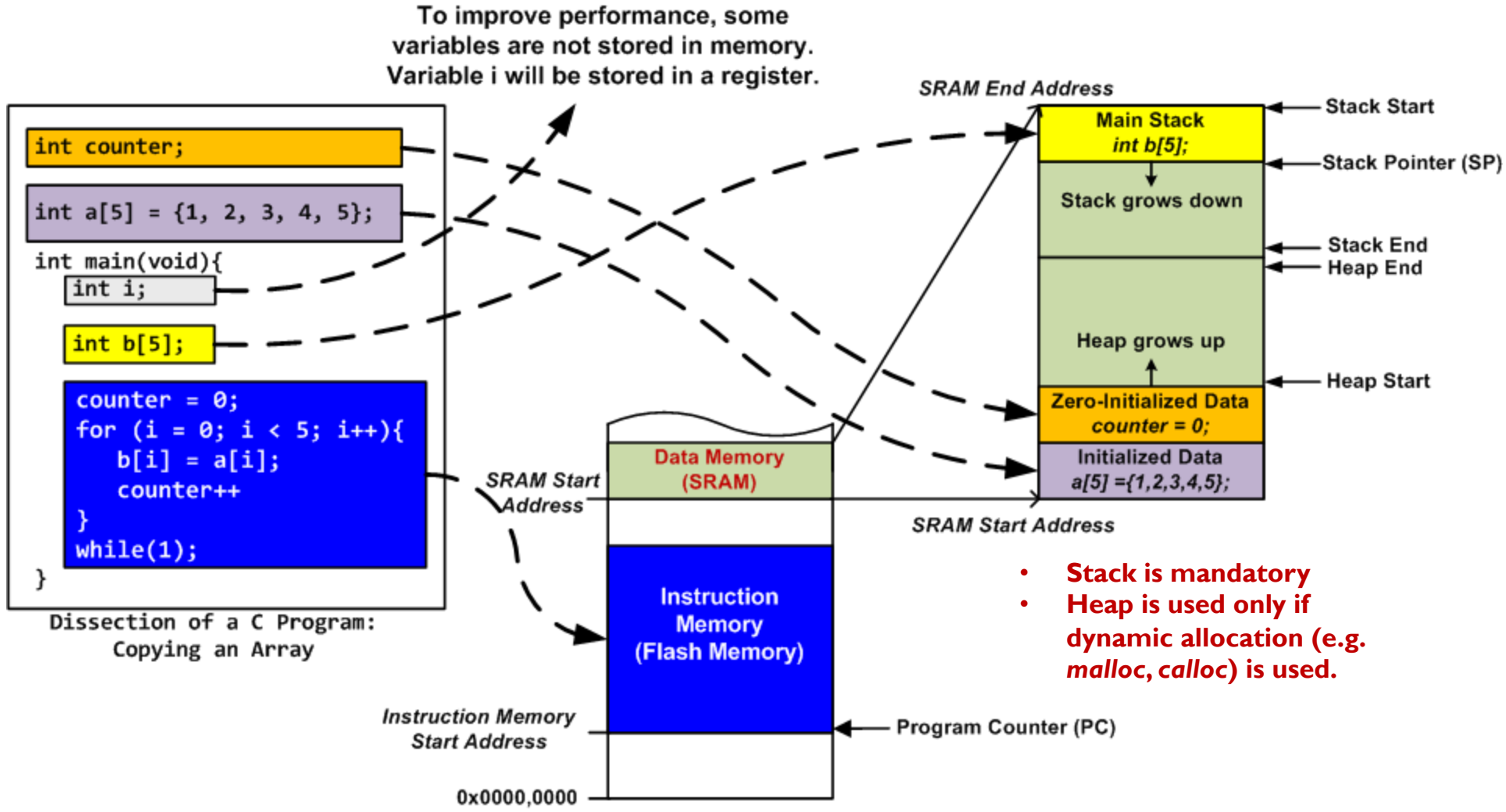




Loading Code and Data into Memory

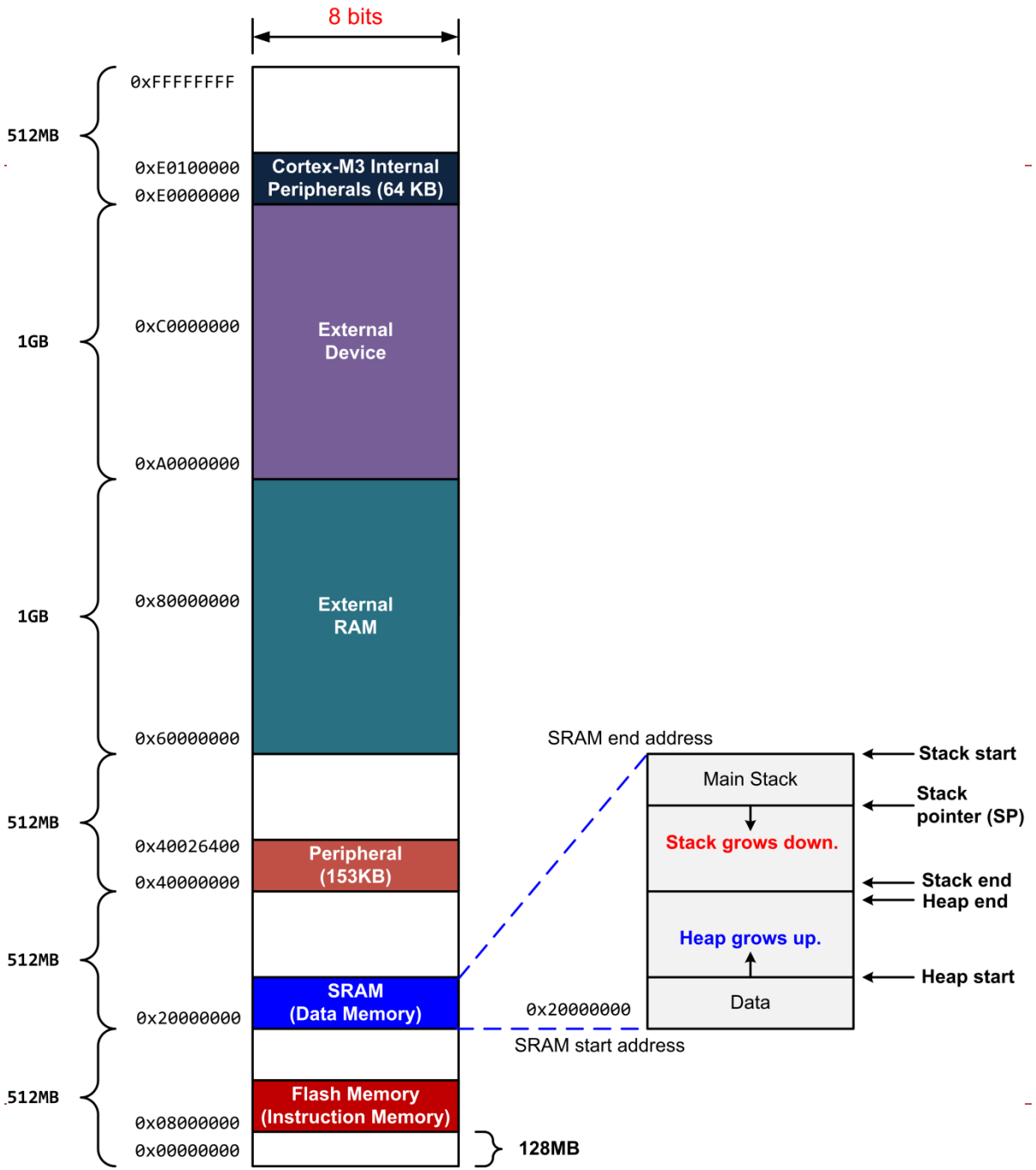


Loading Code and Data into Memory



- **Stack is mandatory**
- **Heap is used only if dynamic allocation (e.g. *malloc*, *calloc*) is used.**

Memory Map


8 bits
0xFFFFFFFF
512MB
0xE0100000
0xE0000000
Cortex-M3 Internal Peripherals (64 KB)
1GB
0xC0000000
External Device
0xA0000000
1GB
0x80000000
External RAM
0x60000000
512MB
0x40026400
0x40000000
Peripheral (153KB)
512MB
0x20000000
SRAM (Data Memory)
512MB
0x08000000
0x00000000
Flash Memory (Instruction Memory)
128MB
SRAM end address
Main Stack
Stack start
Stack pointer (SP)
Stack grows down.
Stack end
Heap end
Heap grows up.
Heap start
Data
0x20000000
SRAM start address